

SCA

Société du Courant Alternatif

Code Source Complet

Électricité générale • Solutions solaires • Matériel électrique
Haut-Katanga, RDC

Stack : React + TypeScript + Tailwind CSS + Supabase

Fichiers inclus : 40 | Lignes de code : 4,833

Généré le 18 septembre 2026

Table des matières

■ Configuration

- index.html
- tailwind.config.js
- vite.config.ts
- tsconfig.json

■ Configuration / PWA

- public/manifest.json
- public/sw.js

■ Types & Base de données

- src/types/index.ts
- src/db/supabase.ts

■ Contextes React

- src/contextes/CartContext.tsx
- src/contextes/AuthContext.tsx

■ Services & Utilitaires

- src/services/api.ts
- src/lib/helpers.ts
- src/lib/pdfGenerator.ts
- src/lib/utills.ts

■ Application Shell

- src/App.tsx
- src/routes.tsx
- src/main.tsx
- src/index.css

■ Composants & Layouts

- src/components/layouts/Navbar.tsx
- src/components/layouts/Footer.tsx
- src/components/layouts/MainLayout.tsx
- src/components/ProductCard.tsx
- src/components/dropzone.tsx

■ Pages Publiques

- src/pages/HomePage.tsx
- src/pages/BoutiquePage.tsx
- src/pages/ProductDetailPage.tsx
- src/pages/CartPage.tsx
- src/pages/ServicesPage.tsx
- src/pages/DevisPage.tsx
- src/pages/ContactPage.tsx
- src/pages/ConseilPage.tsx

■ Espace Administrateur

- src/pages/admin/AdminLoginPage.tsx
- src/pages/admin/AdminLayout.tsx
- src/pages/admin/AdminDashboard.tsx

src/pages/admin/AdminProducts.tsx
src/pages/admin/AdminStock.tsx
src/pages/admin/AdminOrders.tsx
src/pages/admin/AdminInvoices.tsx
src/pages/admin/AdminQuotes.tsx
src/pages/admin/AdminClients.tsx

● Configuration

index.html

26 lignes | 1,399 octets

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" href="https://miaoda-conversation-file.s3cdn.medo.dev/user-ehup8m9cd98g/app-ehusx188iha9/20260918/1/...>
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>SCA - Société du Courant Alternatif | Électricité & Solaire, Haut-Katanga</title>
    <meta name="description" content="SCA - Votre partenaire en électricité générale et solutions solaires au Haut-Katanga">
    <meta name="theme-color" content="#0201a" />
    <link rel="manifest" href="/manifest.json" />
    <meta property="og:title" content="SCA - Société du Courant Alternatif" />
    <meta property="og:description" content="Électricité générale • Solutions solaires • Matériel électrique - Haut-Katanga">
    <meta property="og:image" content="https://miaoda-conversation-file.s3cdn.medo.dev/user-ehup8m9cd98g/app-ehusx188iha9/20260918/1/...>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
    <script>
      if ('serviceWorker' in navigator) {
        window.addEventListener('load', () => {
          navigator.serviceWorker.register('/sw.js').catch(() => {});
        });
      }
    </script>
  </body>
</html>
```

tailwind.config.js

173 lignes | 6,599 octets

```
import tailwindAnimate from 'tailwindcss-animate';
import containerQuery from '@tailwindcss/container-queries';
import intersect from 'tailwindcss-intersect';

export default {
  darkMode: ['class'],
  content: [
    './index.html',
    './pages/**/*.ts,tsx',
    './components/**/*.ts,tsx',
    './app/**/*.ts,tsx',
    './src/**/*.ts,tsx',
    './node_modules/streamdown/dist/**/*.js'
  ],
  safelist: ['border', 'border-border'],
  prefix: '',
  theme: {
    container: {
      center: true,
      padding: '2rem',
      screens: {
        '2xl': '1400px'
      }
    },
  },
  extend: {
```

```

colors: {
  border: 'hsl(var(--border))',
  borderColor: {
    border: 'hsl(var(--border))'
  },
  input: 'hsl(var(--input))',
  ring: 'hsl(var(--ring))',
  background: 'hsl(var(--background))',
  foreground: 'hsl(var(--foreground))',
  primary: {
    DEFAULT: 'hsl(var(--primary))',
    foreground: 'hsl(var(--primary-foreground))'
  },
  secondary: {
    DEFAULT: 'hsl(var(--secondary))',
    foreground: 'hsl(var(--secondary-foreground))'
  },
  destructive: {
    DEFAULT: 'hsl(var(--destructive))',
    foreground: 'hsl(var(--destructive-foreground))'
  },
  muted: {
    DEFAULT: 'hsl(var(--muted))',
    foreground: 'hsl(var(--muted-foreground))'
  },
  accent: {
    DEFAULT: 'hsl(var(--accent))',
    foreground: 'hsl(var(--accent-foreground))'
  },
  popover: {
    DEFAULT: 'hsl(var(--popover))',
    foreground: 'hsl(var(--popover-foreground))'
  },
  card: {
    DEFAULT: 'hsl(var(--card))',
    foreground: 'hsl(var(--card-foreground))'
  },
  education: {
    blue: 'hsl(var(--education-blue))',
    green: 'hsl(var(--education-green))'
  },
  success: 'hsl(var(--success))',
  warning: 'hsl(var(--warning))',
  info: 'hsl(var(--info))',
  sidebar: {
    DEFAULT: 'hsl(var(--sidebar-background))',
    background: 'hsl(var(--sidebar-background))',
    foreground: 'hsl(var(--sidebar-foreground))',
    primary: 'hsl(var(--sidebar-primary))',
    'primary-foreground': 'hsl(var(--sidebar-primary-foreground))',
    accent: 'hsl(var(--sidebar-accent))',
    'accent-foreground': 'hsl(var(--sidebar-accent-foreground))',
    border: 'hsl(var(--sidebar-border))',
    ring: 'hsl(var(--sidebar-ring))'
  },
  chart: {
    '1': 'hsl(var(--chart-1))',
    '2': 'hsl(var(--chart-2))',
    '3': 'hsl(var(--chart-3))',
    '4': 'hsl(var(--chart-4))',
    '5': 'hsl(var(--chart-5))'
  }
},
borderRadius: {
  lg: 'var(--radius)',
  md: 'calc(var(--radius) - 2px)',
  sm: 'calc(var(--radius) - 4px)'
},

```

```

    backgroundImage: {
      'gradient-primary': 'var(--gradient-primary)',
      'gradient-card': 'var(--gradient-card)',
      'gradient-background': 'var(--gradient-background)'
    },
    boxShadow: {
      card: 'var(--shadow-card)',
      hover: 'var(--shadow-hover)'
    },
    keyframes: {
      'accordion-down': {
        from: {
          height: '0'
        },
        to: {
          height: 'var(--radix-accordion-content-height)'
        }
      },
      'accordion-up': {
        from: {
          height: 'var(--radix-accordion-content-height)'
        },
        to: {
          height: '0'
        }
      },
      'fade-in': {
        from: {
          opacity: '0',
          transform: 'translateY(10px)'
        },
        to: {
          opacity: '1',
          transform: 'translateY(0)'
        }
      },
      'slide-in': {
        from: {
          opacity: '0',
          transform: 'translateX(-20px)'
        },
        to: {
          opacity: '1',
          transform: 'translateX(0)'
        }
      }
    },
    animation: {
      'accordion-down': 'accordion-down 0.2s ease-out',
      'accordion-up': 'accordion-up 0.2s ease-out',
      'fade-in': 'fade-in 0.5s ease-out forwards',
      'slide-in': 'slide-in 0.5s ease-out forwards'
    }
  },
  plugins: [
    tailwindAnimate,
    containerQuery,
    intersect,
    function ({addUtilities}) {
      addUtilities(
        {
          '.border-t-solid': {'border-top-style': 'solid'},
          '.border-r-solid': {'border-right-style': 'solid'},
          '.border-b-solid': {'border-bottom-style': 'solid'},
          '.border-l-solid': {'border-left-style': 'solid'},
          '.border-t-dashed': {'border-top-style': 'dashed'},
          '.border-r-dashed': {'border-right-style': 'dashed'},

```

```

        '.border-b-dashed': {'border-bottom-style': 'dashed'},
        '.border-l-dashed': {'border-left-style': 'dashed'},
        '.border-t-dotted': {'border-top-style': 'dotted'},
        '.border-r-dotted': {'border-right-style': 'dotted'},
        '.border-b-dotted': {'border-bottom-style': 'dotted'},
        '.border-l-dotted': {'border-left-style': 'dotted'},
    },
    ['responsive']
  );
},
],
};

```

vite.config.ts

31 lignes | 590 octets

```

import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import svgr from "vite-plugin-svgr";
import path from "path";

export default defineConfig({
  plugins: [
    react(),
    svgr({
      svgrOptions: {
        icon: true,
        exportType: "named",
        namedExport: "ReactComponent",
      },
    }),
  ],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
  optimizeDeps: {
    include: [
      "react",
      "react-dom",
      "react-dom/client",
      "react/jsx-runtime",
      "react/jsx-dev-runtime",
    ],
  },
});

```

tsconfig.json

18 lignes | 370 octets

```

{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["/src/*"]
    }
  }
}

```

```
},
"files": [],
"references": [
  { "path": "../tsconfig.app.json" },
  { "path": "../tsconfig.node.json" }
]
}
```

● Configuration / PWA

public/manifest.json

27 lignes | 844 octets

```
{
  "name": "SCA – Société du Courant Alternatif",
  "short_name": "SCA",
  "description": "Électricité générale • Solutions solaires • Matériel électrique",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0f2240",
  "theme_color": "#c0201a",
  "orientation": "portrait-primary",
  "icons": [
    {
      "src": "https://miaoda-conversation-file.s3cdn.medo.dev/user-ehup8m9cd98g/app-ehusx188iha9/20260918/logo.jpeg",
      "sizes": "192x192",
      "type": "image/jpeg",
      "purpose": "any maskable"
    },
    {
      "src": "https://miaoda-conversation-file.s3cdn.medo.dev/user-ehup8m9cd98g/app-ehusx188iha9/20260918/logo.jpeg",
      "sizes": "512x512",
      "type": "image/jpeg",
      "purpose": "any maskable"
    }
  ],
  "categories": ["shopping", "utilities"],
  "lang": "fr",
  "scope": "/"
}
```

public/sw.js

39 lignes | 1,112 octets

```
// Service Worker SCA
const CACHE_NAME = 'sca-v1';
const STATIC_ASSETS = ['/', '/index.html'];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => cache.addAll(STATIC_ASSETS))
  );
  self.skipWaiting();
});

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((keys) =>
```

```

        Promise.all(keys.filter((k) => k !== CACHE_NAME).map((k) => caches.delete(k)))
    )
    );
    self.clients.claim();
});

self.addEventListener('fetch', (event) => {
    if (event.request.method !== 'GET') return;
    const url = new URL(event.request.url);
    // Network first for API calls
    if (url.hostname.includes('supabase')) {
        event.respondWith(fetch(event.request).catch(() => caches.match(event.request)));
        return;
    }
    // Cache first for static assets
    event.respondWith(
        caches.match(event.request).then(
            (cached) => cached || fetch(event.request).then((res) => {
                const clone = res.clone();
                caches.open(CACHE_NAME).then((c) => c.put(event.request, clone));
                return res;
            })
        )
    );
});

```

● Types & Base de données

[src/types/index.ts](#)

57 lignes | 2,633 octets

```

// SCA types
export type UserRole = 'user' | 'admin';
export type StockStatus = 'en_stock' | 'stock_faible' | 'rupture';
export type OrderStatus = 'en_attente' | 'confirme' | 'en_preparation' | 'livre' | 'annule';
export type DeliveryMode = 'livraison' | 'retrait';
export type QuoteStatus = 'nouveau' | 'en_cours' | 'traite' | 'rejete';

export interface Profile {
    id: string; email: string | null; phone: string | null;
    full_name: string | null; role: UserRole; created_at: string;
}
export interface Category {
    id: string; name: string; slug: string; description: string | null;
    type: 'electricite' | 'solaire'; icon: string | null; sort_order: number; created_at: string;
}
export interface Product {
    id: string; category_id: string; name: string; description: string | null;
    reference: string; price: number; stock_quantity: number; stock_threshold: number;
    image_url: string | null; is_active: boolean; created_at: string; updated_at: string;
    categories?: Category;
}
export interface Client {
    id: string; full_name: string; phone: string; email: string | null;
    address: string | null; profile_id: string | null; created_at: string;
}
export interface Order {
    id: string; order_number: string; client_id: string; profile_id: string | null;
    status: OrderStatus; delivery_mode: DeliveryMode; delivery_address: string | null;
    subtotal: number; delivery_fee: number; total: number; notes: string | null;
    created_at: string; updated_at: string;
}

```

```

    clients?: Client; order_items?: OrderItem[];
  }
export interface OrderItem {
  id: string; order_id: string; product_id: string; product_name: string;
  product_reference: string; unit_price: number; quantity: number; total: number; created_at: string;
  products?: Product;
}
export interface Invoice {
  id: string; invoice_number: string; order_id: string; client_id: string;
  total: number; issued_at: string; created_at: string;
  orders?: Order; clients?: Client;
}
export interface Quote {
  id: string; quote_number: string; full_name: string; phone: string; email: string | null;
  project_type: string; description: string; address: string | null;
  photo_urls: string[] | null; status: QuoteStatus; admin_notes: string | null;
  created_at: string; updated_at: string;
}
export interface StudyRequest {
  id: string; full_name: string; phone: string; email: string | null;
  need_description: string; status: string; created_at: string;
}
export interface Review {
  id: string; author_name: string; rating: number; comment: string;
  is_published: boolean; created_at: string;
}
export interface CartItem { product: Product; quantity: number; }

```

src/db/supabase.ts

8 lignes | 324 octets

```

import { createClient } from "@supabase/supabase-js";

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseAnonKey);

```

● Contextes React

src/contexts/CartContext.tsx

72 lignes | 2,304 octets

```

import React, { createContext, useContext, useEffect, useState, useCallback } from 'react';
import type { CartItem, Product } from '@types/index';

interface CartContextValue {
  items: CartItem[];
  count: number;
  total: number;
  addItem: (product: Product, qty?: number) => void;
  removeItem: (productId: string) => void;
  updateQty: (productId: string, qty: number) => void;
  clearCart: () => void;
}

const CartContext = createContext<CartContextValue | null>(null);
export const CartProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {

```

```

const [items, setItems] = useState<CartItem[]>(() => {
  try {
    return JSON.parse(sessionStorage.getItem('sca_cart') || '[]');
  } catch { return []; }
});

useEffect(() => {
  sessionStorage.setItem('sca_cart', JSON.stringify(items));
}, [items]);

const addItem = useCallback((product: Product, qty = 1) => {
  setItems((prev) => {
    const existing = prev.find((i) => i.product.id === product.id);
    if (existing) {
      return prev.map((i) =>
        i.product.id === product.id
          ? { ...i, quantity: Math.min(i.quantity + qty, product.stock_quantity) }
          : i
      );
    }
    return [...prev, { product, quantity: Math.min(qty, product.stock_quantity) }];
  });
}, []);

const removeItem = useCallback((productId: string) => {
  setItems((prev) => prev.filter((i) => i.product.id !== productId));
}, []);

const updateQty = useCallback((productId: string, qty: number) => {
  if (qty &lt;= 0) { removeItem(productId); return; }
  setItems((prev) =>
    prev.map((i) =>
      i.product.id === productId
        ? { ...i, quantity: Math.min(qty, i.product.stock_quantity) }
        : i
    )
  );
}, [removeItem]);

const clearCart = useCallback(() => setItems([], []));

const count = items.reduce((s, i) => s + i.quantity, 0);
const total = items.reduce((s, i) => s + i.quantity * i.product.price, 0);

return (
  <CartContext.Provider value={{ items, count, total, addItem, removeItem, updateQty, clearCart }}>
    {children}
  </CartContext.Provider>
);
};

export const useCart = () => {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart doit être utilisé dans CartProvider');
  return ctx;
};

```

[src/contexts/AuthContext.tsx](#)

131 lignes | 3,588 octets

```

import { createContext, useContext, useEffect, useState, type ReactNode } from 'react';
// @ts-ignore
import { supabase } from '@db/supabase';
import type { User } from '@supabase/supabase-js';
// @ts-ignore

```

```

import type { Profile } from '@types/types';
import { toast } from 'sonner';

export async function getProfile(userId: string): Promise<Profile | null>; {
  const { data, error } = await supabase
    .from('profiles')
    .select('*')
    .eq('id', userId)
    .maybeSingle();

  if (error) {
    console.error('■■■■■■■■■■:', error);
    return null;
  }
  return data;
}

interface AuthContextType {
  user: User | null;
  profile: Profile | null;
  loading: boolean;
  signInWithUsername: (username: string, password: string) => Promise<{ error: Error | null }>;
  signUpWithUsername: (username: string, password: string) => Promise<{ error: Error | null }>;
  signOut: () => Promise<void>;
  refreshProfile: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [profile, setProfile] = useState<Profile | null>(null);
  const [loading, setLoading] = useState(true);

  const refreshProfile = async () => {
    if (!user) {
      setProfile(null);
      return;
    }

    const profileData = await getProfile(user.id);
    setProfile(profileData);
  };

  useEffect(() => {
    supabase
      .auth
      .getSession()
      // @ts-ignore
      .then(({ data: { session } }) => {
        setUser(session?.user ?? null);
        if (session?.user) {
          getProfile(session.user.id).then(setProfile);
        }
      })
      // @ts-ignore
      .catch(error => {
        toast.error(`■■■■■■■■■■: ${error.message}`);
      })
      .finally(() => {
        setLoading(false);
      });

    // @ts-ignore
    // In this function, do NOT use any await calls. Use `.then()` instead to avoid deadlocks.
    const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, session) => {
      setUser(session?.user ?? null);
      if (session?.user) {
        getProfile(session.user.id).then(setProfile);
      }
    });
  }, []);
}

```

```

        } else {
            setProfile(null);
        }
    });

    return () => subscription.unsubscribe();
}, []);

const signInWithUsername = async (username: string, password: string) => {
    try {
        const email = `${username}@miaoda.com`;
        const { error } = await supabase.auth.signInWithPassword({
            email,
            password,
        });

        if (error) throw error;
        return { error: null };
    } catch (error) {
        return { error: error as Error };
    }
};

const signUpWithUsername = async (username: string, password: string) => {
    try {
        const email = `${username}@miaoda.com`;
        const { error } = await supabase.auth.signUp({
            email,
            password,
        });

        if (error) throw error;
        return { error: null };
    } catch (error) {
        return { error: error as Error };
    }
};

const signOut = async () => {
    await supabase.auth.signOut();
    setUser(null);
    setProfile(null);
};

return (
    <AuthContext.Provider value={{ user, profile, loading, signInWithUsername, signUpWithUsername, signOut, refreshProfi
        {children}
    </AuthContext.Provider>
);
}

export function useAuth() {
    const context = useContext(AuthContext);
    if (context === undefined) {
        throw new Error('useAuth must be used within an AuthProvider');
    }
    return context;
}

```

● Services & Utilitaires

[illegible]

[illegible]

```
export async function updateOrderStatus(id: string, status: Order['status']): Promise<void> {
  const { error } = await supabase.from('orders').update({ status, updated_at: new Date().toISOString() }).eq('id', id);
  if (error) throw error;
}
```

```
export async function fetchInvoiceByOrderId(orderId: string): Promise<Invoice | null> {
  const { data, error } = await supabase
    .from('invoices')
    .select('*', orders!order_id(*, clients!client_id(*), order_items(*))')
    .eq('order_id', orderId)
    .maybeSingle();
  if (error) throw error;
  return data;
}
```

```
export async function fetchInvoices(limit = 50, offset = 0): Promise<Invoice[]> {
  const { data, error } = await supabase
    .from('invoices')
    .select('*', clients!client_id(*), orders!order_id(*)')
    .order('created_at', { ascending: false })
    .range(offset, offset + limit - 1);
  if (error) throw error;
  return Array.isArray(data) ? data : [];
}
```

```
export async function submitQuote(quote: Omit<Quote, 'id' | 'quote_number' | 'status' | 'admin_notes' | 'created_at' | 'updated_at'>, { id, quote_number }) {
  const { error } = await supabase.from('quotes').insert({ ...quote, quote_number });
  if (error) throw error;
}
```

```
export async function fetchQuotes(limit = 50, offset = 0): Promise<Quote[]> {
  const { data, error } = await supabase
    .from('quotes')
    .select('*')
    .order('created_at', { ascending: false })
    .range(offset, offset + limit - 1);
  if (error) throw error;
  return Array.isArray(data) ? data : [];
}
```

```
export async function updateQuoteStatus(id: string, status: QuoteStatus, admin_notes?: string): Promise<void> {
  const { error } = await supabase
    .from('quotes')
    .update({ status, admin_notes: admin_notes || null, updated_at: new Date().toISOString() })
    .eq('id', id);
  if (error) throw error;
}
```

```
export async function submitStudyRequest(req: Omit<StudyRequest, 'id' | 'status' | 'created_at'>): Promise<void> {
  const { error } = await supabase.from('study_requests').insert(req);
  if (error) throw error;
}
```

```
export async function fetchStudyRequests(limit = 50): Promise<StudyRequest[]> {
  const { data, error } = await supabase
    .from('study_requests')
    .select('*')
    .order('created_at', { ascending: false })
    .limit(limit);
  if (error) throw error;
  return Array.isArray(data) ? data : [];
}
```

[illegible]

```
export async function fetchReviews(): Promise<Review[]> {
  const { data, error } = await supabase
    .from('reviews')
    .select('*')
    .eq('is_published', true)
    .order('created_at', { ascending: false })
    .limit(20);
  if (error) throw error;
  return Array.isArray(data) ? data : [];
}

// █ Clients ████████████████████████████████████████████████████████████
export async function fetchClients(limit = 50, offset = 0): Promise<Client[]> {
  const { data, error } = await supabase
    .from('clients')
    .select('*')
    .order('created_at', { ascending: false })
    .range(offset, offset + limit - 1);
  if (error) throw error;
  return Array.isArray(data) ? data : [];
}

// █ Dashboard Stats ████████████████████████████████████████████████████████████
export async function fetchDashboardStats() {
  const today = new Date().toISOString().slice(0, 10);
  const [ordersToday, totalOrders, pendingQuotes, clients, products] = await Promise.all([
    supabase.from('orders').select('total', { count: 'exact' }).gte('created_at', today).eq('status', 'confirme'),
    supabase.from('orders').select('id', { count: 'exact' }),
    supabase.from('quotes').select('id', { count: 'exact' }).eq('status', 'nouveau'),
    supabase.from('clients').select('id', { count: 'exact' }),
    supabase.from('products').select('id, stock_quantity, stock_threshold', { count: 'exact' }),
  ]);

  const todaySales = (ordersToday.data || []).reduce((s: number, o: { total: number }) => s + (o.total || 0), 0);
  const prods = Array.isArray(products.data) ? products.data : [];
  const enStock = prods.filter((p) => p.stock_quantity >= p.stock_threshold).length;
  const stockFaible = prods.filter((p) => p.stock_quantity < 0 && p.stock_quantity <= p.stock_threshold).length;
  const rupture = prods.filter((p) => p.stock_quantity === 0).length;

  return {
    todaySales,
    totalOrders: totalOrders.count || 0,
    pendingQuotes: pendingQuotes.count || 0,
    totalClients: clients.count || 0,
    totalProducts: products.count || 0,
    enStock,
    stockFaible,
    rupture,
  };
}
```

src/lib/helpers.ts

```
import { type Product, type StockStatus } from '@types/index';

export function getStockStatus(product: Product): StockStatus {
  if (product.stock_quantity === 0) return 'rupture';
  if (product.stock_quantity <= product.stock_threshold) return 'stock_faible';
  return 'en_stock';
}

export function stockLabel(status: StockStatus): string {
  const labels: Record<StockStatus, string> = {
    en_stock: 'En stock',
    stock_faible: 'Stock faible',
  };
  return labels[status];
}
```

```

    rupture: 'Rupture de stock',
  };
  return labels[status];
}

export function stockBadgeClass(status: StockStatus): string {
  const classes: Record<StockStatus, string> = {
    en_stock: 'badge-en-stock',
    stock_faible: 'badge-stock-faible',
    rupture: 'badge-rupture',
  };
  return classes[status];
}

export function formatPrice(price: number): string {
  return `${price.toFixed(2)} $`;
}

export function formatDate(dateStr: string): string {
  return new Date(dateStr).toLocaleDateString('fr-FR', {
    year: 'numeric', month: 'long', day: 'numeric',
  });
}

export function orderStatusLabel(status: string): string {
  const labels: Record<string, string> = {
    en_attente: 'En attente',
    confirme: 'Confirmé',
    en_preparation: 'En préparation',
    livre: 'Livré',
    annule: 'Annulé',
  };
  return labels[status] || status;
}

export function quoteStatusLabel(status: string): string {
  const labels: Record<string, string> = {
    nouveau: 'Nouveau',
    en_cours: 'En cours',
    traite: 'Traité',
    rejete: 'Rejeté',
  };
  return labels[status] || status;
}

```

src/lib/pdfGenerator.ts

117 lignes | 4,311 octets

```

// Utilitaire PDF - génération de factures SCA
import jsPDF from 'jspdf';
import autoTable from 'jspdf-autotable';
import type { Invoice } from '@types/index';

export function generateInvoicePDF(invoice: Invoice): void {
  const doc = new jsPDF({ orientation: 'portrait', unit: 'mm', format: 'a4' });
  const order = invoice.orders;
  const client = invoice.orders?.clients || invoice.clients;
  const items = order?.order_items || [];

  // ■■ Header band ■■
  doc.setFillColor(15, 34, 64); // deep navy
  doc.rect(0, 0, 210, 38, 'F');

  // Logo placeholder text
  doc.setTextColor(255, 255, 255);

```

```

doc.setFontSize(22);
doc.setFont('helvetica', 'bold');
doc.text('SCA', 14, 18);
doc.setFontSize(9);
doc.setFont('helvetica', 'normal');
doc.text('Société du Courant Alternatif', 14, 25);
doc.text('Électricité générale • Solutions solaires • Matériel électrique', 14, 31);

// Red accent stripe
doc.setFillColor(192, 32, 26);
doc.rect(0, 38, 210, 2, 'F');

// ■■ Invoice title ■■
doc.setTextColor(192, 32, 26);
doc.setFontSize(18);
doc.setFont('helvetica', 'bold');
doc.text('FACTURE', 140, 54);
doc.setTextColor(40, 40, 40);
doc.setFontSize(10);
doc.setFont('helvetica', 'normal');
doc.text(`N° ${invoice.invoice_number}`, 140, 62);
doc.text(`Date : ${new Date(invoice.issued_at).toLocaleDateString('fr-FR')}`, 140, 69);

// ■■ Client info ■■
doc.setFontSize(10);
doc.setFont('helvetica', 'bold');
doc.text('FACTURÉ À', 14, 54);
doc.setFont('helvetica', 'normal');
if (client) {
  doc.text(client.full_name || '', 14, 62);
  doc.text(`Tél: ${client.phone || ''}`, 14, 69);
  if (client.email) doc.text(`Email: ${client.email}`, 14, 76);
  if (client.address) doc.text(`Adresse: ${client.address}`, 14, 83);
}

// ■■ Order meta ■■
const yMeta = 92;
doc.setFont('helvetica', 'bold');
doc.text(`Commande N° ${order?.order_number || ''}`, 14, yMeta);
doc.setFont('helvetica', 'normal');
doc.text(`Mode : ${order?.delivery_mode === 'livraison' ? 'Livraison à domicile' : 'Retrait en magasin'}`, 14, yMeta + 7);

// ■■ Items table ■■
autoTable(doc, {
  startY: yMeta + 16,
  head: [['Réf', 'Désignation', 'Qté', 'Prix Unit.', 'Total']],
  body: items.map((it) => [
    it.product_reference,
    it.product_name,
    it.quantity.toString(),
    `${Number(it.unit_price).toFixed(2)} $`,
    `${Number(it.total).toFixed(2)} $`,
  ]),
  headStyles: { fillColor: [15, 34, 64], textColor: 255, fontStyle: 'bold' },
  alternateRowStyles: { fillColor: [245, 247, 250] },
  columnStyles: {
    0: { cellWidth: 28 },
    1: { cellWidth: 80 },
    2: { halign: 'center', cellWidth: 18 },
    3: { halign: 'right', cellWidth: 30 },
    4: { halign: 'right', cellWidth: 30 },
  },
  styles: { fontSize: 9, cellPadding: 3 },
  margin: { left: 14, right: 14 },
});

// ■■ Totals ■■
// eslint-disable-next-line @typescript-eslint/no-explicit-any

```

```

const finalY = (doc as any).lastAutoTable.finalY + 6;
const totalsX = 130;
doc.setFontSize(9);
doc.setFont('helvetica', 'normal');
doc.text('Sous-total :', totalsX, finalY);
doc.text(`${Number(order?.subtotal || 0).toFixed(2)} $`, 195, finalY, { align: 'right' });

doc.text('Livraison :', totalsX, finalY + 7);
doc.text(`${Number(order?.delivery_fee || 0).toFixed(2)} $`, 195, finalY + 7, { align: 'right' });

// Total line
doc.setFillColors(15, 34, 64);
doc.rect(totalsX - 2, finalY + 12, 67, 8, 'F');
doc.setFont('helvetica', 'bold');
doc.setTextColor(255, 255, 255);
doc.setFontSize(10);
doc.text('TOTAL :', totalsX + 1, finalY + 18);
doc.text(`${Number(invoice.total).toFixed(2)} $`, 193, finalY + 18, { align: 'right' });

// ■■ Footer ■■
doc.setTextColor(120, 120, 120);
doc.setFont('helvetica', 'normal');
doc.setFontSize(8);
const pageH = doc.internal.pageSize.height;
doc.setFillColors(240, 242, 245);
doc.rect(0, pageH - 20, 210, 20, 'F');
doc.setTextColor(80, 80, 80);
doc.text('SCA - Société du Courant Alternatif | Haut-Katanga, RDC', 105, pageH - 13, { align: 'center' });
doc.text('■ 085 865 74 75 / 097 528 31 55', 105, pageH - 7, { align: 'center' });

doc.save(`${invoice.invoice_number}.pdf`);
}

```

src/lib/utills.ts

39 lignes | 974 octets

```

import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export type Params = Partial<
  Record<keyof URLSearchParams, string | number | null | undefined>
>;

export function createQueryString(
  params: Params,
  searchParams: URLSearchParams
) {
  const newSearchParams = new URLSearchParams(searchParams?.toString());

  for (const [key, value] of Object.entries(params)) {
    if (value === null || value === undefined) {
      newSearchParams.delete(key);
    } else {
      newSearchParams.set(key, String(value));
    }
  }

  return newSearchParams.toString();
}

export function formatDate(

```

```
    date: Date | string | number,
    opts: Intl.DateTimeFormatOptions = {}
  ) {
    return new Intl.DateTimeFormat("zh-CN", {
      month: opts.month ?? "long",
      day: opts.day ?? "numeric",
      year: opts.year ?? "numeric",
      ...opts,
    }).format(new Date(date));
  }
}
```

● Application Shell

src/App.tsx

25 lignes | 780 octets

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import IntersectObserver from '@components/common/IntersectObserver';
import { Toaster } from '@components/ui/sonner';
import { CartProvider } from '@contexts/CartContext';
import { routes } from './routes';

const App: React.FC = () => {
  return (
    <Router>
      <CartProvider>
        <IntersectObserver />
        <Routes>
          {routes.map((route, index) => (
            <Route key={index} path={route.path} element={route.element} />
          ))}
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
        <Toaster richColors position="top-right" />
      </CartProvider>
    </Router>
  );
};

export default App;
```

src/routes.tsx

49 lignes | 3,041 octets

```
import React from 'react';
import type { ReactNode } from 'react';

import HomePage from './pages/HomePage';
import BoutiquePage from './pages/BoutiquePage';
import ProductDetailPage from './pages/ProductDetailPage';
import CartPage from './pages/CartPage';
import ServicesPage from './pages/ServicesPage';
import DevisPage from './pages/DevisPage';
import ContactPage from './pages/ContactPage';
import ConseilPage from './pages/ConseilPage';
import AdminLoginPage from './pages/admin/AdminLoginPage';
import AdminDashboard from './pages/admin/AdminDashboard';
```



```

/* stylelint-disable */
@tailwind base;
@tailwind components;
@tailwind utilities;

/* SCA – Société du Courant Alternatif
Design: Industrial-Professional · Red/Blue/White brand palette
Bold structural identity: deep navy panels, brand red accent,
crisp white surfaces, angular industrial confidence */

@font-face {
  font-family: 'AlibabaPuHuiTi';
  src: url('https://resource-static.bj.bcebos.com/fonts-skill/AlibabaPuHuiTi-Regular.ttf') format('truetype');
  font-weight: 400;
  font-display: swap;
}

@font-face {
  font-family: 'AlibabaPuHuiTi';
  src: url('https://resource-static.bj.bcebos.com/fonts-skill/AlibabaPuHuiTi-Bold.ttf') format('truetype');
  font-weight: 700;
  font-display: swap;
}

@layer base {
  :root {
    --radius: 0.375rem;

    /* Crisp white background */
    --background: 210 20% 98%;
    --foreground: 220 30% 12%;

    /* Light card surface */
    --card: 0 0% 100%;
    --card-foreground: 220 30% 12%;
    --popover: 0 0% 100%;
    --popover-foreground: 220 30% 12%;

    /* SCA Brand Red – primary action */
    --primary: 4 86% 46%;
    --primary-foreground: 0 0% 100%;

    /* Deep Navy – structural/secondary */
    --secondary: 220 55% 20%;
    --secondary-foreground: 0 0% 100%;

    /* Neutral muted */
    --muted: 210 20% 93%;
    --muted-foreground: 220 15% 45%;

    /* Electric Blue accent */
    --accent: 213 85% 42%;
    --accent-foreground: 0 0% 100%;

    --destructive: 0 75% 50%;
    --destructive-foreground: 0 0% 100%;

    --border: 220 15% 85%;
    --input: 220 15% 85%;
    --ring: 4 86% 46%;

    /* Semantic status colors */
    --success: 142 70% 40%;
    --warning: 38 92% 50%;
    --info: 213 85% 42%;
    --success-foreground: 0 0% 100%;
    --warning-foreground: 0 0% 100%;
    --info-foreground: 0 0% 100%;
  }
}

```

```

--chart-1: 4 86% 46%;
--chart-2: 213 85% 42%;
--chart-3: 220 55% 20%;
--chart-4: 142 70% 40%;
--chart-5: 38 92% 50%;

/* Sidebar — deep navy */
--sidebar-background: 220 55% 14%;
--sidebar-foreground: 210 30% 88%;
--sidebar-primary: 4 86% 52%;
--sidebar-primary-foreground: 0 0% 100%;
--sidebar-accent: 220 45% 22%;
--sidebar-accent-foreground: 0 0% 100%;
--sidebar-border: 220 40% 22%;
--sidebar-ring: 4 86% 52%;
}

.dark {
--background: 220 30% 8%;
--foreground: 210 20% 92%;
--card: 220 30% 11%;
--card-foreground: 210 20% 92%;
--popover: 220 30% 11%;
--popover-foreground: 210 20% 92%;
--primary: 4 86% 52%;
--primary-foreground: 0 0% 100%;
--secondary: 220 45% 24%;
--secondary-foreground: 0 0% 100%;
--muted: 220 30% 16%;
--muted-foreground: 220 15% 58%;
--accent: 213 85% 50%;
--accent-foreground: 0 0% 100%;
--destructive: 0 75% 55%;
--destructive-foreground: 0 0% 100%;
--border: 220 25% 22%;
--input: 220 25% 22%;
--ring: 4 86% 52%;
--success: 142 60% 45%;
--warning: 38 85% 55%;
--info: 213 75% 55%;
--chart-1: 4 86% 52%;
--chart-2: 213 75% 55%;
--chart-3: 220 45% 50%;
--chart-4: 142 60% 45%;
--chart-5: 38 85% 55%;
--sidebar-background: 220 35% 8%;
--sidebar-foreground: 210 25% 85%;
--sidebar-primary: 4 86% 52%;
--sidebar-primary-foreground: 0 0% 100%;
--sidebar-accent: 220 35% 16%;
--sidebar-accent-foreground: 0 0% 100%;
--sidebar-border: 220 30% 18%;
--sidebar-ring: 4 86% 52%;
}
}

@layer base {
* {
@apply border-border;
scrollbar-width: thin;
scrollbar-color: hsl(var(--border)) transparent;
}

html { color-scheme: light dark; }

body {
@apply bg-background text-foreground;
font-family: 'AlibabaPuHuiTi', 'Segoe UI', system-ui, sans-serif;

```

```

    font-size: 16px;
    line-height: 1.6;
}

h1, h2, h3, h4, h5, h6 {
    font-family: 'AlibabaPuHuiTi', 'Segoe UI', system-ui, sans-serif;
    font-weight: 700;
    line-height: 1.25;
    letter-spacing: -0.01em;
}

*::-webkit-scrollbar { width: 6px; height: 6px; }
*::-webkit-scrollbar-thumb { background: hsl(var(--border)); border-radius: 3px; }
}

@layer components {
    /* SCA brand diagonal stripe – core visual signature */
    .sca-stripe {
        background-image: repeating-linear-gradient(
            -45deg,
            transparent,
            transparent 8px,
            hsl(var(--primary) / 0.08) 8px,
            hsl(var(--primary) / 0.08) 10px
        );
    }

    /* Red underline accent on headings */
    .sca-heading-accent {
        position: relative;
        display: inline-block;
    }
    .sca-heading-accent::after {
        content: '';
        position: absolute;
        left: 0;
        bottom: -6px;
        width: 48px;
        height: 3px;
        background: hsl(var(--primary));
        border-radius: 2px;
    }

    /* Electric bolt decoration */
    .sca-bolt-bg {
        background-image:
            radial-gradient(circle at 80% 20%, hsl(var(--primary) / 0.12) 0%, transparent 50%),
            radial-gradient(circle at 20% 80%, hsl(var(--accent) / 0.10) 0%, transparent 45%);
    }

    /* Product card hover lift */
    .product-card {
        transition: transform 200ms ease, box-shadow 200ms ease, border-color 200ms ease;
    }
    .product-card:hover {
        transform: translateY(-3px);
        box-shadow: 0 8px 24px hsl(var(--primary) / 0.12);
        border-color: hsl(var(--primary) / 0.4);
    }

    /* Nav link active indicator */
    .nav-link-active {
        color: hsl(var(--primary));
        font-weight: 600;
    }

    /* Stock badge */
    .badge-en-stock { @apply bg-green-100 text-green-700 border-green-200; }
}

```

```

.badge-stock-faible { @apply bg-orange-100 text-orange-700 border-orange-200; }
.badge-rupture { @apply bg-red-100 text-red-700 border-red-200; }

/* Hero gradient */
.hero-gradient {
  background: linear-gradient(135deg,
    hsl(220 55% 14%) 0%,
    hsl(220 55% 20%) 50%,
    hsl(213 85% 28%) 100%);
}

/* Section divider */
.section-divider {
  height: 4px;
  background: linear-gradient(90deg,
    hsl(var(--primary)) 0%,
    hsl(var(--accent)) 100%);
}

/* Gradient text utility */
.gradient-text {
  background-image: linear-gradient(90deg, hsl(var(--primary)), hsl(var(--accent)));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

/* Smooth page transitions */
@keyframes fadeInUp {
  from { opacity: 0; transform: translateY(16px); }
  to { opacity: 1; transform: translateY(0); }
}
.animate-fade-in-up {
  animation: fadeInUp 0.4s ease forwards;
}

```

● Composants & Layouts

src/components/layouts/Navbar.tsx

138 lignes | 6,066 octets

```

import React, { useState } from 'react';
import { Link, useLocation } from 'react-router-dom';
import { ShoppingCart, Menu, X, Zap, Phone } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Sheet, SheetContent, SheetTrigger } from '@components/ui/sheet';
import { Badge } from '@components/ui/badge';
import { useCart } from '@contexts/CartContext';

const navLinks = [
  { label: 'Accueil', path: '/' },
  { label: 'Boutique', path: '/boutique' },
  { label: 'Services', path: '/services' },
  { label: 'Devis', path: '/devis' },
  { label: 'Contact', path: '/contact' },
];

const Navbar: React.FC = () => {
  const { count } = useCart();
  const location = useLocation();

```

```

const [open, setOpen] = useState(false);

const isActive = (path: string) =>{
  path === '/' ? location.pathname === '/' : location.pathname.startsWith(path);
}

return (
  <header className="sticky top-0 z-50 w-full border-b border-border bg-secondary shadow-md">
    {/* Top strip */}
    <div className="bg-primary hidden md:flex items-center justify-end px-6 py-1 text-xs text-primary-foreground gap-4">
      <span className="flex items-center gap-1">&Phone className="w-3 h-3" /> 085 865 74 75</span>
      <span className="flex items-center gap-1">&Phone className="w-3 h-3" /> 097 528 31 55</span>
    </div>

    {/* Main nav */}
    <div className="flex h-16 items-center justify-between px-4 md:px-8 max-w-7xl mx-auto w-full">
      {/* Logo */}
      <Link to="/" className="flex items-center gap-3 shrink-0">
        
        <div className="hidden sm:block">
          <p className="font-bold text-primary-foreground text-sm leading-tight">SCA</p>
          <p className="text-xs text-secondary-foreground/70 leading-tight">Courant Alternatif</p>
        </div>
      </Link>

      {/* Desktop links */}
      <nav className="hidden md:flex items-center gap-1">
        {navLinks.map((link) => {
          <Link
            key={link.path}
            to={link.path}
            className={`px-4 py-2 text-sm font-medium rounded-md transition-colors ${
              isActive(link.path)
                ? 'bg-primary text-primary-foreground'
                : 'text-secondary-foreground/80 hover:bg-secondary-foreground/10 hover:text-white'
            }`}
          >{link.label}</Link>
        })}
        <Link to="/admin" className="px-4 py-2 text-sm font-medium text-secondary-foreground/60 hover:text-white transition-colors">Admin</Link>
      </nav>

      {/* Cart + mobile menu */}
      <div className="flex items-center gap-2">
        <Link to="/panier">
          <Button variant="ghost" size="icon" className="relative text-secondary-foreground/80 hover:text-white hover:scale-110">
            <ShoppingCart className="h-5 w-5" />
            {count > 0 & & {
              <Badge className="absolute -top-1 -right-1 h-5 w-5 p-0 flex items-center justify-center bg-primary text-white text-xs">
                {count > 99 ? '99+' : count}
              </Badge>
            }
          </Button>
        </Link>

        {/* Mobile hamburger */}
        <Sheet open={open} onOpenChange={setOpen}>
          <SheetTrigger asChild>
            <Button variant="ghost" size="icon" className="md:hidden text-secondary-foreground/80 hover:text-white">
              <Menu className="h-5 w-5" />
            </Button>
          </SheetTrigger>
          <SheetContent side="left" className="w-72 bg-sidebar p-0">

```

```

    <div className="flex items-center gap-3 p-4 border-b border-sidebar-border">
      
    </div>
    <p className="font-bold text-sidebar-foreground">SCA</p>
    <p className="text-xs text-sidebar-foreground/60">Courant Alternatif</p>
  </div>
  <nav className="flex flex-col p-4 gap-1">
    {navLinks.map((link) => {
      <Link
        key={link.path}
        to={link.path}
        onClick={() => setOpen(false)}
        className={`flex items-center gap-3 px-4 py-3 rounded-md text-sm font-medium transition-colors ${
          isActive(link.path)
            ? 'bg-sidebar-primary text-sidebar-primary-foreground'
            : 'text-sidebar-foreground hover:bg-sidebar-accent'
        }`}
      >
        {link.label}
      </Link>
    })}
    <Link
      to="/admin"
      onClick={() => setOpen(false)}
      className="flex items-center gap-3 px-4 py-3 rounded-md text-sm font-medium text-sidebar-foreground/60 hover:bg-sidebar-accent"
    >
      Administration
    </Link>
  </nav>
  <div className="p-4 border-t border-sidebar-border mt-auto">
    <p className="text-xs text-sidebar-foreground/50 flex items-center gap-1">
      <Zap className="w-3 h-3 text-sidebar-primary" />
      Haut-Katanga, RDC
    </p>
    <p className="text-xs text-sidebar-foreground/50 mt-1">085 865 74 75</p>
  </div>
  <SheetContent>
  </SheetContent>
</div>
</div>
</header>
);
};

export default Navbar;

```

91 lignes | 3,999 octets

```


</div>
<p className="font-bold text-white">SCA</p>
<p className="text-xs text-secondary-foreground/60">Société du Courant Alternatif</p>
</div>
</div>
<p className="text-sm text-secondary-foreground/70 leading-relaxed">
  Votre partenaire de confiance en électricité générale et solutions solaires au Haut-Katanga.
</p>
<div className="flex items-center gap-1 mt-3 text-xs text-secondary-foreground/50">
  <Zap className="w-3 h-3 text-primary" />
  Disponibilité · Qualité · Accessibilité
</div>
</div>

{/* Navigation */}
<div>
  <h3 className="font-semibold text-white mb-4">Navigation</h3>
  <ul className="space-y-2 text-sm text-secondary-foreground/70">
    <li>
      <div>
        {
          label: 'Accueil', path: '/' },
          label: 'Boutique', path: '/boutique' },
          label: 'Nos services', path: '/services' },
          label: 'Demander un devis', path: '/devis' },
          label: 'Conseil / Étude', path: '/conseil' },
          label: 'Contact', path: '/contact' },
        ].map((l) => (
          <li key={l.path}>
            <Link to={l.path} className="hover:text-primary transition-colors">
              {l.label}
            </Link>
          </li>
        ))
      </ul>
    </div>

    {/* Contact */}
    <div>
      <h3 className="font-semibold text-white mb-4">Contact</h3>
      <ul className="space-y-3 text-sm text-secondary-foreground/70">
        <li className="flex items-start gap-2">
          <MapPin className="w-4 h-4 mt-0.5 text-primary shrink-0" />
          Haut-Katanga, République Démocratique du Congo
        </li>
        <li className="flex items-center gap-2">
          <Phone className="w-4 h-4 text-primary shrink-0" />
          <a href="tel:+243858657475" className="hover:text-primary transition-colors">085 865 74 75</a>
        </li>
        <li className="flex items-center gap-2">
          <Phone className="w-4 h-4 text-primary shrink-0" />
          <a href="tel:+243975283155" className="hover:text-primary transition-colors">097 528 31 55</a>
        </li>
        <li className="flex items-center gap-2">
          <MessageCircle className="w-4 h-4 text-primary shrink-0" />
          <a href="https://wa.me/243858657475" target="_blank" rel="noreferrer" className="hover:text-primary transition-colors">
            WhatsApp SCA
          </a>
        </li>
        <li className="flex items-center gap-2">
          <Mail className="w-4 h-4 text-primary shrink-0" />
          <a href="mailto:contact@sca-katanga.com" className="hover:text-primary transition-colors">
            contact@sca-katanga.com
          </a>
        </li>
      </ul>
    </div>
  </div>

```

```

        </ul>
      </div>
    </div>

    <div className="border-t border-secondary-foreground/10 px-4 md:px-8 py-4 max-w-7xl mx-auto flex flex-col sm:flex-row"
      <span>© {new Date().getFullYear()} SCA - Société du Courant Alternatif. Tous droits réservés.</span>
      <span>Haut-Katanga, RDC</span>
    </div>
  </footer>
);

export default Footer;

```

src/components/layouts/MainLayout.tsx

13 lignes | 349 octets

```

import React from 'react';
import Navbar from '../Navbar';
import Footer from '../Footer';

const MainLayout: React.FC<{ children: React.ReactNode }> = ({ children }) => (
  <div className="flex flex-col min-h-screen w-full">
    <Navbar />
    <main className="flex-1 min-w-0">{children}</main>
    <Footer />
  </div>
);

export default MainLayout;

```

src/components/ProductCard.tsx

105 lignes | 3,716 octets

```

import React from 'react';
import { Link } from 'react-router-dom';
import { ShoppingCart, Package, Star, Eye } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Badge } from '@components/ui/badge';
import type { Product } from '@types/index';
import { getStockStatus, stockLabel, stockBadgeClass, formatPrice } from '@lib/helpers';
import { useCart } from '@contexts/CartContext';
import { toast } from 'sonner';

interface ProductCardProps {
  product: Product;
}

const ProductCard: React.FC<ProductCardProps> = ({ product }) => {
  const { addItem } = useCart();
  const status = getStockStatus(product);

  const handleAdd = (e: React.MouseEvent) => {
    e.preventDefault();
    if (status === 'rupture') return;
    addItem(product, 1);
    toast.success(`{product.name} ajouté au panier`);
  };

  return (
    <Link to={`/produit/${product.id}`} className="block">
      <div className="product-card bg-card border border-border rounded-lg overflow-hidden h-full flex flex-col">

```

```

    { /* Image */
    <div className="aspect-[4/3] w-full overflow-hidden bg-muted relative">
      {product.image_url ? (
        <img
          src={product.image_url}
          alt={product.name}
          className="w-full h-full object-cover"
        />
      ) : (
        <div className="w-full h-full flex items-center justify-center">
          <Package className="w-12 h-12 text-muted-foreground" />
        </div>
      )}
    { /* Stock badge overlay */
    <div className="absolute top-2 right-2">
      <Badge variant="outline" className={`text-xs border ${stockBadgeClass(status)}`>
        {stockLabel(status)}
      </Badge>
    </div>
    </div>

    { /* Content */
    <div className="flex flex-col flex-1 p-4 gap-2">
      <p className="text-xs text-muted-foreground font-mono">{product.reference}</p>
      <h3 className="font-semibold text-foreground text-sm leading-snug line-clamp-2 flex-1">
        {product.name}
      </h3>
      {product.description & & (
        <p className="text-xs text-muted-foreground line-clamp-2">{product.description}</p>
      )}

      <div className="flex items-center justify-between mt-auto pt-2 border-t border-border">
        <span className="text-lg font-bold text-primary">{formatPrice(product.price)}</span>
        <div className="flex gap-1">
          <Button
            variant="outline"
            size="icon"
            className="h-8 w-8"
            onClick={(e) => { e.preventDefault(); }}
            asChild
          >
            <Link to={`/produit/${product.id}`}>
              <Eye className="h-3.5 w-3.5" />
            </Link>
          </Button>
          <Button
            size="icon"
            className="h-8 w-8 bg-primary hover:bg-primary/90"
            onClick={handleAdd}
            disabled={status === 'rupture'}
          >
            <ShoppingCart className="h-3.5 w-3.5" />
          </Button>
        </div>
      </div>
    </div>
    </div>
    </div>
    </div>
  </Link>
);
};

export default ProductCard;

// Star rating component
export const StarRating = React.FC<{ rating: number; size?: 'sm' | 'md' }> = ({ rating, size = 'md' }) => {
  const sz = size === 'sm' ? 'w-3.5 h-3.5' : 'w-5 h-5';
  return (
    <div className="flex items-center gap-0.5">

```

```

    {[1, 2, 3, 4, 5].map((i) => (
      <Star
        key={i}
        className={` ${sz} ${i} <= rating ? 'text-yellow-400 fill-yellow-400' : 'text-muted-foreground'}`}
      />
    ))}
  </div>
);
};

```

src/components/dropzone.tsx

227 lignes | 7,615 octets

```

import { cn } from '@lib/Utils'
import { type UseSupabaseUploadReturn } from '@hooks/use-supabase-upload'
import { Button } from '@components/ui/button'
import { CheckCircle, File, Loader2, Upload, X } from 'lucide-react'
import { createContext, type PropsWithChildren, useCallback, useContext } from 'react'

export const formatBytes = (
  bytes: number,
  decimals = 2,
  size?: 'bytes' | 'KB' | 'MB' | 'GB' | 'TB' | 'PB' | 'EB' | 'ZB' | 'YB'
) => {
  const k = 1000
  const dm = decimals < 0 ? 0 : decimals
  const sizes = ['bytes', 'KB', 'MB', 'GB', 'TB', 'PB', 'EB', 'ZB', 'YB']

  if (bytes === 0 || bytes === undefined) return size !== undefined ? `0 ${size}` : '0 bytes'
  const i = size !== undefined ? sizes.indexOf(size) : Math.floor(Math.log(bytes) / Math.log(k))
  return parseFloat((bytes / Math.pow(k, i)).toFixed(dm)) + ' ' + sizes[i]
}

type DropzoneContextType = Omit<UseSupabaseUploadReturn, 'getRootProps' | 'getInputProps'>

const DropzoneContext = createContext<DropzoneContextType | undefined>(undefined)

type DropzoneProps = UseSupabaseUploadReturn & {
  className?: string
}

const Dropzone = ({
  className,
  children,
  getRootProps,
  getInputProps,
  ...restProps
}: PropsWithChildren<DropzoneProps>) => {
  const isSuccess = restProps.isSuccess
  const isActive = restProps.isDragActive
  const isInvalid =
    (restProps.isDragActive && restProps.isDragReject) ||
    (restProps.errors.length > 0 && !restProps.isSuccess) ||
    restProps.files.some((file) => file.errors.length !== 0)

  return (
    <DropzoneContext.Provider value={{ ...restProps }}>
      <div
        {...getRootProps({
          className: cn(
            'border-2 border-gray-300 rounded-lg p-6 text-center bg-card transition-colors duration-300 text-foreground',
            className,
            isSuccess ? 'border-solid' : 'border-dashed',
            isActive && 'border-primary bg-primary/10',
            isInvalid && 'border-destructive bg-destructive/10'
          )
        })}
      >

```

```

    ),
  ]))
  <input {...getInputProps()} />
  {children}
  </div>
  </DropzoneContext.Provider>
)
}
const DropzoneContent = ({ className }: { className?: string }) => {
  const {
    files,
    setFiles,
    onUpload,
    loading,
    successes,
    errors,
    maxFileSize,
    maxFiles,
    isSuccess,
  } = useDropzoneContext()

  const exceedMaxFiles = files.length > maxFiles

  const handleRemoveFile = useCallback(
    (fileName: string) => {
      setFiles(files.filter((file) => file.name !== fileName))
    },
    [files, setFiles]
  )

  if (isSuccess) {
    return (
      <div className={cn('flex flex-row items-center gap-x-2 justify-center', className)}>
        <CheckCircle size={16} className="text-primary" />
        <p className="text-primary text-sm">
          Successfully uploaded {files.length} file{files.length > 1 ? 's' : ''}
        </p>
      </div>
    )
  }

  return (
    <div className={cn('flex flex-col', className)}>
      {files.map((file, idx) => {
        const fileError = errors.find((e) => e.name === file.name)
        const isSuccessfullyUploaded = !!successes.find((e) => e === file.name)

        return (
          <div
            key={`_${file.name}-${idx}`}
            className="flex items-center gap-x-4 border-b py-2 first:mt-4 last:mb-4 "
          >
            {file.type.startsWith('image/') ? (
              <div className="h-10 w-10 rounded border overflow-hidden shrink-0 bg-muted flex items-center justify-center">
                <img src={file.preview} alt={file.name} className="object-cover" />
              </div>
            ) : (
              <div className="h-10 w-10 rounded border bg-muted flex items-center justify-center">
                <File size={18} />
              </div>
            )
          }

          <div className="shrink grow flex flex-col items-start truncate">
            <p title={file.name} className="text-sm truncate max-w-full">
              {file.name}
            </p>
            {file.errors.length > 0 ? (

```

```

        <p className="text-xs text-destructive">
          {file.errors
            .map((e) =>
              e.message.startsWith('File is larger than')
                ? `File is larger than ${formatBytes(maxFileSize, 2)} (Size: ${formatBytes(file.size, 2)})`
                : e.message
            )
            .join(', ')}
        </p>
      ) : loading && !isSuccessfullyUploaded ? (
        <p className="text-xs text-muted-foreground">Uploading file...</p>
      ) : !!fileError ? (
        <p className="text-xs text-destructive">Failed to upload: {fileError.message}</p>
      ) : isSuccessfullyUploaded ? (
        <p className="text-xs text-primary">Successfully uploaded file</p>
      ) : (
        <p className="text-xs text-muted-foreground">{formatBytes(file.size, 2)}</p>
      )
    </div>

    {!loading && !isSuccessfullyUploaded && (
      <Button
        size="icon"
        variant="link"
        className="shrink-0 justify-self-end text-muted-foreground hover:text-foreground"
        onClick={() => handleRemoveFile(file.name)}
      >
        <X />
      </Button>
    )}
  </div>
)
}}
{exceedMaxFiles && (
  <p className="text-sm text-left mt-2 text-destructive">
    You may upload only up to {maxFiles} files, please remove {files.length - maxFiles} file
    {files.length - maxFiles > 1 ? 's' : ''}.
  </p>
)}
{files.length > 0 && !exceedMaxFiles && (
  <div className="mt-2">
    <Button
      variant="outline"
      onClick={onUpload}
      disabled={files.some((file) => file.errors.length !== 0) || loading}
    >
      {loading ? (
        <&Loader2 className="mr-2 h-4 w-4 animate-spin" />
        Uploading...
      ) : (
        <&gt;Upload files</>
      )}
    </Button>
  </div>
)}
</div>
)
}

const DropzoneEmptyState = ({ className }: { className?: string }) => {
  const { maxFiles, maxFileSize, inputRef, isSuccess } = useDropzoneContext()

  if (isSuccess) {
    return null
  }

  return (

```

```

<div className={cn('flex flex-col items-center gap-y-2', className)}>
  <Upload size={20} className="text-muted-foreground" />
  <p className="text-sm">
    Upload{!maxFiles & & maxFiles & 1 ? ` ${maxFiles} ` : ''} file
    {!maxFiles || maxFiles & 1 ? 's' : ''}
  </p>
  <div className="flex flex-col items-center gap-y-1">
    <p className="text-xs text-muted-foreground">
      Drag and drop or{' '}
      <a
        onClick={() => inputRef.current?.click()}
        className="underline cursor-pointer transition hover:text-foreground"
      >
        select {maxFiles === 1 ? `file` : `files`}
      </a>{' '}
      to upload
    </p>
    {maxFileSize !== Number.POSITIVE_INFINITY & & (
      <p className="text-xs text-muted-foreground">
        Maximum file size: {formatBytes(maxFileSize, 2)}
      </p>
    )}
  </div>
</div>
)
}

const useDropzoneContext = () => {
  const context = useContext(DropzoneContext)

  if (!context) {
    throw new Error('useDropzoneContext must be used within a Dropzone')
  }

  return context
}

export { Dropzone, DropzoneContent, DropzoneEmptyState, useDropzoneContext }

```

src/pages/HomePage.tsx

[illegible]

[illegible]


```

        </Button>
      </CardContent>
    </Card>
  )}
  </div>
</section>

{ /* Featured products */
<section className="bg-muted/40">
  <div className="max-w-7xl mx-auto px-4 md:px-8 py-16">
    <div className="flex items-center justify-between mb-10">
      <div>
        <h2 className="text-2xl md:text-3xl font-bold sca-heading-accent mb-6">
          Produits Vedettes
        </h2>
        <p className="text-muted-foreground">Matériel électrique et solaire de qualité.</p>
      </div>
      <Button asChild variant="outline">
        <Link to="/boutique">Voir tout →</Link>
      </Button>
    </div>

    {loading ? (
      <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
        {[...Array(6)].map((_, i) => (
          <div key={i} className="bg-muted animate-pulse rounded-lg aspect-[3/4]" />
        ))}
      </div>
    ) : (
      <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
        {featuredProducts.map((p) => <ProductCard key={p.id} product={p} />)}
      </div>
    )}
    </div>
  </section>

  { /* CTA banner */
  <section className="hero-gradient">
    <div className="max-w-7xl mx-auto px-4 md:px-8 py-14 text-center">
      <Sun className="w-10 h-10 text-primary mx-auto mb-4" />
      <h2 className="text-2xl md:text-3xl font-bold text-white mb-4 text-balance">
        Vous ne savez pas quoi choisir ?
      </h2>
      <p className="text-secondary-foreground/70 mb-8 max-w-xl mx-auto">
        Décrivez votre projet et nos experts SCA vous proposent une étude personnalisée et gratuite.
      </p>
      <div className="flex flex-wrap gap-4 justify-center">
        <Button asChild size="lg" className="bg-primary hover:bg-primary/90 text-primary-foreground">
          <Link to="/conseil">Demander une étude gratuite</Link>
        </Button>
        <Button asChild size="lg" variant="ghost" className="border border-white/30 text-white hover:bg-white/10">
          <Link to="/devis">Demander un devis</Link>
        </Button>
      </div>
    </div>
  </section>

  { /* Reviews */
  <reviews.length > 0 & & (
    <section className="max-w-7xl mx-auto px-4 md:px-8 py-16">
      <h2 className="text-2xl md:text-3xl font-bold sca-heading-accent mb-10">
        Avis de nos clients
      </h2>
      <div className="grid md:grid-cols-3 gap-6">
        {reviews.slice(0, 6).map((r) => (
          <Card key={r.id} className="border border-border">
            <CardContent className="p-5">
              <StarRating rating={r.rating} />
              <p className="text-sm text-muted-foreground mt-3 italic leading-relaxed">

```

```

        "{r.comment}"
        </p>
        <p className="text-sm font-semibold text-foreground mt-3">{r.author_name}</p>
        </CardContent>
        </Card>
    )}
    </div>
    </section>
  )}
  </MainLayout>
);
};

export default HomePage;

```

src/pages/BoutiquePage.tsx

178 lignes | 7,204 octets

```

import React, { useEffect, useState, useCallback } from 'react';
import { Search, Filter, SlidersHorizontal } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Badge } from '@components/ui/badge';
import MainLayout from '@components/layouts/MainLayout';
import ProductCard from '@components/ProductCard';
import { fetchCategories, fetchProducts } from '@services/api';
import type { Category, Product } from '@types/index';

const BoutiquePage: React.FC = () => {
  const [categories, setCategories] = useState<Category[]>([]);
  const [products, setProducts] = useState<Product[]>([]);
  const [selectedCat, setSelectedCat] = useState<string>('all');
  const [selectedType, setSelectedType] = useState<string>('all');
  const [search, setSearch] = useState('');
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchCategories().then(setCategories).catch(console.error);
  }, []);

  const load = useCallback(() => {
    setLoading(true);
    fetchProducts({
      categoryId: selectedCat !== 'all' ? selectedCat : undefined,
      search: search.trim() || undefined,
      limit: 60,
    })
      .then((prods) => {
        const filtered = selectedType !== 'all'
          ? prods.filter((p) => p.categories?.type === selectedType)
          : prods;
        setProducts(filtered);
      })
      .catch(console.error)
      .finally(() => setLoading(false));
  }, [selectedCat, selectedType, search]);

  useEffect(() => { load(); }, [load]);

  const electriciteCategories = categories.filter((c) => c.type === 'electricite');
  const solaireCategories = categories.filter((c) => c.type === 'solaire');

  return (
    <MainLayout>
      { /* Header */ }
    </MainLayout>
  );
};

```

```

<div className="hero-gradient border-b border-border">
  <div className="max-w-7xl mx-auto px-4 md:px-8 py-10">
    <h1 className="text-3xl font-bold text-white mb-2">Catalogue</h1>
    <p className="text-secondary-foreground/70">Matériel électrique et solutions solaires</p>
  </div>
</div>

<div className="max-w-7xl mx-auto px-4 md:px-8 py-8">
  {/* Search */}
  <div className="flex gap-3 mb-6">
    <div className="relative flex-1 min-w-0">
      <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
      <Input
        placeholder="Rechercher un produit..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        className="pl-10"
      />
    </div>
    <div>
      <Button variant="outline" onClick={load} className="shrink-0">
        <SlidersHorizontal className="w-4 h-4 mr-2" />
        Rechercher
      </Button>
    </div>
  </div>

  <div className="flex flex-col md:flex-row gap-8">
    {/* Sidebar filters */}
    <aside className="md:w-56 shrink-0">
      <div className="bg-card border border-border rounded-lg p-4 sticky top-24">
        <h2 className="font-semibold text-foreground mb-3 flex items-center gap-2">
          <Filter className="w-4 h-4 text-primary" /> Filtres
        </h2>
        {/* All */}
        <Button
          variant={selectedCat === 'all' & selectedType === 'all' ? 'default' : 'ghost'}
          size="sm"
          className="w-full justify-start mb-2"
          onClick={() => { setSelectedCat('all'); setSelectedType('all'); }}
        >
          Tous les produits
        </Button>

        {/* Électricité group */}
        <p className="text-xs font-semibold text-primary uppercase tracking-widest mt-4 mb-2 px-2">
          ■ Électricité
        </p>
        <Button
          variant={selectedType === 'electricite' & selectedCat === 'all' ? 'default' : 'ghost'}
          size="sm"
          className="w-full justify-start mb-1"
          onClick={() => { setSelectedType('electricite'); setSelectedCat('all'); }}
        >
          Tout – Électricité
        </Button>
        {electriciteCategories.map((c) => (
          <Button
            key={c.id}
            variant={selectedCat === c.id ? 'default' : 'ghost'}
            size="sm"
            className="w-full justify-start mb-1 text-xs"
            onClick={() => { setSelectedCat(c.id); setSelectedType('electricite'); }}
          >
            {c.name}
          </Button>
        ))}

        {/* Solaire group */}
        <p className="text-xs font-semibold text-accent uppercase tracking-widest mt-4 mb-2 px-2">

```

```

        ■■ Solaire
    </p>
    <Button
        variant={selectedType === 'solaire' && selectedCat === 'all' ? 'default' : 'ghost'}
        size="sm"
        className="w-full justify-start mb-1"
        onClick={() => { setSelectedType('solaire'); setSelectedCat('all'); }}
    >
        Tout – Solaire
    </Button>
    {solaireCategories.map((c) => (
        <Button
            key={c.id}
            variant={selectedCat === c.id ? 'default' : 'ghost'}
            size="sm"
            className="w-full justify-start mb-1 text-xs"
            onClick={() => { setSelectedCat(c.id); setSelectedType('solaire'); }}
        >
            {c.name}
        </Button>
    ))}
    </div>
</aside>

{/* Products grid */}
<div className="flex-1 min-w-0">
    <div className="flex items-center justify-between mb-4">
        <p className="text-sm text-muted-foreground">
            {loading ? 'Chargement...' : `${products.length} produit${products.length !== 1 ? 's' : ''}`}
        </p>
        {selectedCat !== 'all' && (
            <Badge variant="secondary" className="cursor-pointer" onClick={() => setSelectedCat('all')}>
                {categories.find((c) => c.id === selectedCat)?.name} ×
            </Badge>
        )}
    </div>

    {loading ? (
        <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
            {[...Array(9)].map((_, i) => (
                <div key={i} className="bg-muted animate-pulse rounded-lg aspect-[3/4]" />
            ))}
        </div>
    ) : products.length === 0 ? (
        <div className="text-center py-20 text-muted-foreground">
            <Search className="w-12 h-12 mx-auto mb-4 opacity-30" />
            <p className="text-lg font-medium">Aucun produit trouvé</p>
            <p className="text-sm">Essayez une autre recherche ou catégorie</p>
        </div>
    ) : (
        <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
            {products.map((p) => <ProductCard key={p.id} product={p} />)}
        </div>
    )}
    </div>
</div>
</div>
</MainLayout>
);
};

export default BoutiquePage;

```

src/pages/ProductDetailPage.tsx

180 lignes | 6,643 octets

```
import React, { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import { ShoppingCart, ChevronLeft, Package, Minus, Plus } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Badge } from '@components/ui/badge';
import { Separator } from '@components/ui/separator';
import MainLayout from '@components/layouts/MainLayout';
import { fetchProductById } from '@services/api';
import { getStockStatus, stockLabel, stockBadgeClass, formatPrice } from '@lib/helpers';
import { useCart } from '@contexts/CartContext';
import { toast } from 'sonner';
import type { Product } from '@types/index';

const ProductDetailPage: React.FC = () => {
  const { id } = useParams<{ id: string }>();
  const [product, setProduct] = useState<Product | null>(null);
  const [loading, setLoading] = useState(true);
  const [qty, setQty] = useState(1);
  const { addItem } = useCart();

  useEffect(() => {
    if (!id) return;
    setLoading(true);
    fetchProductById(id)
      .then(setProduct)
      .catch(console.error)
      .finally(() => setLoading(false));
  }, [id]);

  if (loading) {
    return (
      <MainLayout>
      <div className="max-w-4xl mx-auto px-4 md:px-8 py-12">
        <div className="grid md:grid-cols-2 gap-8">
          <div className="bg-muted animate-pulse rounded-lg aspect-square" />
          <div className="space-y-4">
            {[...Array(6)].map((_, i) => <div key={i} className="h-4 bg-muted animate-pulse rounded" />)}
          </div>
        </div>
      </div>
      </MainLayout>
    );
  }

  if (!product) {
    return (
      <MainLayout>
      <div className="max-w-4xl mx-auto px-4 py-20 text-center text-muted-foreground">
        <Package className="w-16 h-16 mx-auto mb-4 opacity-30" />
        <p className="text-xl font-semibold">Produit introuvable</p>
        <Button asChild variant="outline" className="mt-6">
          <Link to="/boutique">< Retour à la boutique</Link>
        </Button>
      </div>
      </MainLayout>
    );
  }

  const status = getStockStatus(product);
  const maxQty = Math.min(product.stock_quantity, 99);

  const handleAdd = () => {
    if (status === 'rupture') return;
    addItem(product, qty);
  };
};
```

```

toast.success(`${qty}× ${product.name} ajouté${qty} > 1 ? 's' : ''} au panier`);
};

return (
  <MainLayout>
    <div className="max-w-5xl mx-auto px-4 md:px-8 py-8">
      </* Breadcrumb */>
      <div className="flex items-center gap-2 text-sm text-muted-foreground mb-6">
        <Link to="/" className="hover:text-primary">Accueil</Link>
        <span>/</span>
        <Link to="/boutique" className="hover:text-primary">Boutique</Link>
        {product.categories && (
          <span>
            <span>/</span>
            <span>{product.categories.name}</span>
          </span>
        )}
        <span>/</span>
      </div>
      <div className="text-foreground truncate max-w-[200px]">{product.name}</div>

      <Button asChild variant="ghost" size="sm" className="mb-6 -ml-2">
        <Link to="/boutique">
          <ChevronLeft className="w-4 h-4 mr-1" />
          Retour à la boutique
        </Link>
      </Button>

      <div className="grid md:grid-cols-2 gap-10">
        </* Image */>
        <div className="aspect-square w-full overflow-hidden rounded-lg bg-muted border border-border">
          {product.image_url ? (
            <img src={product.image_url} alt={product.name} className="w-full h-full object-cover" />
          ) : (
            <div className="w-full h-full flex items-center justify-center">
              <Package className="w-24 h-24 text-muted-foreground" />
            </div>
          )}
        </div>

        </* Details */>
        <div className="flex flex-col gap-4">
          {product.categories && (
            <Badge variant="secondary" className="w-fit">
              {product.categories.type === 'electricite' ? '■' : '■■'} {product.categories.name}
            </Badge>
          )}

          <h1 className="text-2xl md:text-3xl font-bold text-foreground text-balance">{product.name}</h1>

          <p className="text-sm font-mono text-muted-foreground">Réf: {product.reference}</p>

          <div className="text-3xl font-bold text-primary">{formatPrice(product.price)}</div>

          <Badge variant="outline" className={`w-fit border ${stockBadgeClass(status)}`}>
            {stockLabel(status)}
          </Badge>

          {product.description && (
            <div>
              <Separator />
              <p className="text-muted-foreground leading-relaxed text-sm">{product.description}</p>
            </div>
          )}

          <Separator />

          </* Qty selector */>

```

```

{status !== 'rupture' && (
  <div className="flex items-center gap-3">
    <span className="text-sm font-medium">Quantité :</span>
    <div className="flex items-center border border-border rounded-md">
      <Button
        variant="ghost"
        size="icon"
        className="h-9 w-9 rounded-none"
        onClick={() => setQty(Math.max(1, qty - 1))}
        disabled={qty <= 1}
      />
      <Minus className="w-4 h-4" />
    </div>
    <span className="w-10 text-center text-sm font-semibold">{qty}</span>
    <Button
      variant="ghost"
      size="icon"
      className="h-9 w-9 rounded-none"
      onClick={() => setQty(Math.min(maxQty, qty + 1))}
      disabled={qty >= maxQty}
    />
    <Plus className="w-4 h-4" />
  </div>
)}

<Button
  size="lg"
  className="bg-primary hover:bg-primary/90 text-primary-foreground gap-2 w-full md:w-auto"
  onClick={handleAdd}
  disabled={status === 'rupture'}
/>
<ShoppingCart className="w-5 h-5" />
{status === 'rupture' ? 'Rupture de stock' : 'Ajouter au panier'}
/>

<Button asChild variant="outline" size="lg" className="w-full md:w-auto">
  <Link to="/devis">Demander un devis pour ce produit</Link>
</Button>
</div>
</div>
</div>
</MainLayout>
);
};

export default ProductDetailPage;

```

src/pages/CartPage.tsx

281 lignes | 13,781 octets

```

import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { Trash2, Minus, Plus, ShoppingCart, ChevronLeft, Truck, Store } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Label } from '@components/ui/label';
import { Separator } from '@components/ui/separator';
import { RadioGroup, RadioGroupItem } from '@components/ui/radio-group';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import MainLayout from '@components/layouts/MainLayout';
import { useCart } from '@contexts/CartContext';
import { formatPrice } from '@lib/helpers';
import { createOrder } from '@services/api';

```

```

import { toast } from 'sonner';
import type { DeliveryMode } from '@types/index';

const DELIVERY_FEE = 5;

const CartPage: React.FC = () => {
  const { items, count, total, removeItem, updateQty, clearCart } = useCart();
  const navigate = useNavigate();

  const [step, setStep] = useState<'cart' | 'checkout' | 'confirmed'>('cart');
  const [deliveryMode, setDeliveryMode] = useState<DeliveryMode>('livraison');
  const [orderId, setOrderId] = useState('');
  const [orderNumber, setOrderNumber] = useState('');
  const [submitting, setSubmitting] = useState(false);

  const [form, setForm] = useState({
    full_name: '', phone: '', email: '', address: '', notes: '',
  });

  const deliveryFee = deliveryMode === 'livraison' ? DELIVERY_FEE : 0;
  const grandTotal = total + deliveryFee;

  const handleOrder = async () => {
    if (!form.full_name.trim() || !form.phone.trim()) {
      toast.error('Veuillez remplir votre nom et téléphone');
      return;
    }
    setSubmitting(true);
    try {
      const orderItems = items.map((i) => ({
        product_id: i.product.id,
        product_name: i.product.name,
        product_reference: i.product.reference,
        unit_price: i.product.price,
        quantity: i.quantity,
        total: i.quantity * i.product.price,
      }));
      const result = await createOrder(
        { full_name: form.full_name, phone: form.phone, email: form.email || null, address: form.address || null },
        orderItems,
        {
          delivery_mode: deliveryMode,
          delivery_address: deliveryMode === 'livraison' ? form.address : undefined,
          subtotal: total,
          delivery_fee: deliveryFee,
          total: grandTotal,
          notes: form.notes || undefined,
        }
      );
      setOrderId(result.orderId);
      setOrderNumber(result.orderNumber);
      clearCart();
      setStep('confirmed');
      toast.success('Commande confirmée !');
    } catch (err) {
      console.error(err);
      toast.error('Erreur lors de la commande. Veuillez réessayer.');
```

```

    } finally {
      setSubmitting(false);
    }
  };

  // ■■ Empty cart ■■
  if (count === 0 && step !== 'confirmed') {
    return (
      <MainLayout>
      <div className="max-w-2xl mx-auto px-4 py-20 text-center">
        <ShoppingCart className="w-20 h-20 mx-auto mb-6 text-muted-foreground opacity-40" />

```

```
<code>&lt;/h1 className="text-2xl font-bold mb-3">Votre panier est vide</h1>
&lt;p className="text-muted-foreground mb-8">Découvrez notre catalogue de matériel électrique et solaire.</p>
&lt;Button asChild size="lg" className="bg-primary hover:bg-primary/90 text-primary-foreground">
  &lt;Link to="/boutique">Voir nos produits</Link>
&lt;/Button>
&lt;/div>
&lt;/MainLayout>
);
}

// ■■■ Confirmed ■■■
if (step === 'confirmed') {
  return (
    &lt;MainLayout>
      &lt;div className="max-w-lg mx-auto px-4 py-20 text-center">
        &lt;div className="w-20 h-20 bg-green-100 rounded-full flex items-center justify-center mx-auto mb-6">
          &lt;span className="text-4xl">■</span>
        &lt;/div>
        &lt;h1 className="text-2xl font-bold text-foreground mb-3">Commande confirmée !</h1>
        &lt;p className="text-muted-foreground mb-2">
          Votre commande &lt;span className="font-bold text-foreground">{orderNumber}</span> a été enregistrée.
        &lt;/p>
        &lt;p className="text-muted-foreground mb-8 text-sm">
          Notre équipe vous contactera sous peu pour confirmer les détails.
        &lt;/p>
        &lt;div className="flex flex-col sm:flex-row gap-3 justify-center">
          &lt;Button asChild size="lg" variant="outline">
            &lt;Link to="/boutique">Continuer mes achats</Link>
          &lt;/Button>
          &lt;Button asChild size="lg" className="bg-primary hover:bg-primary/90 text-primary-foreground">
            &lt;Link to="/">Retour à l'accueil</Link>
          &lt;/Button>
        &lt;/div>
      &lt;/div>
    &lt;/MainLayout>
  );
}

return (
  &lt;MainLayout>
    &lt;div className="max-w-6xl mx-auto px-4 md:px-8 py-8">
      &lt;Button asChild variant="ghost" size="sm" className="mb-6 -ml-2">
        &lt;Link to="/boutique">&lt;ChevronLeft className="w-4 h-4 mr-1" />Continuer mes achats</Link>
      &lt;/Button>

      &lt;h1 className="text-2xl font-bold mb-8">
        {step === 'cart' ? `Mon Panier (${count})` : 'Finaliser la commande'}
      &lt;/h1>

      &lt;div className="grid lg:grid-cols-3 gap-8">
        /* Left: items or checkout form */
        &lt;div className="lg:col-span-2 space-y-4">
          {step === 'cart' ? (
            &lt;&gt;
              {items.map((item) => (
                &lt;div key={item.product.id} className="flex gap-4 bg-card border border-border rounded-lg p-4 items-center">
                  &lt;div className="w-16 h-16 rounded-md overflow-hidden bg-muted shrink-0">
                    {item.product.image_url ? (
                      &lt;img src={item.product.image_url} alt={item.product.name} className="w-full h-full object-cover" />
                    ) : (
                      &lt;div className="w-full h-full flex items-center justify-center">
                        &lt;ShoppingCart className="w-6 h-6 text-muted-foreground" />
                      &lt;/div>
                    )
                  &lt;/div>
                  &lt;div className="flex-1 min-w-0">
                    &lt;p className="font-semibold text-sm text-foreground truncate">{item.product.name}</p>
                    &lt;p className="text-xs text-muted-foreground">{item.product.reference}</p>

```

```
<div></div>  
<div className="flex items-center border border-border rounded-md shrink-0">  
  <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => updateQty(item.product.i  
    <Minus className="w-3 h-3" />  
  </Button>  
  <span className="w-8 text-center text-sm font-semibold">{item.quantity}</span>  
  <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => updateQty(item.product.i  
    <Plus className="w-3 h-3" />  
  </Button>  
</div>  
  
<p className="w-20 text-right font-bold text-sm text-foreground shrink-0">  
  {formatPrice(item.quantity * item.product.price)}  
</p>  
  
<Button variant="ghost" size="icon" className="text-destructive hover:text-destructive/80 shrink-0"  
  <Trash2 className="w-4 h-4" />  
</Button>  
</div>  
)})  
</div>  
) : (  
/* Checkout form */  
<Card className="border border-border">  
  <CardHeader><CardTitle>Vos informations</CardTitle><CardHeader>  
  <CardContent className="space-y-4">  
    <div className="grid sm:grid-cols-2 gap-4">  
      <div className="space-y-2">  
        <Label htmlFor="full_name">Nom complet *</Label>  
        <Input id="full_name" value={form.full_name} onChange={(e) => setForm({ ...form, full_name: e.ta  
      </div>  
      <div className="space-y-2">  
        <Label htmlFor="phone">Téléphone *</Label>  
        <Input id="phone" value={form.phone} onChange={(e) => setForm({ ...form, phone: e.target.value  
      </div>  
    </div>  
    <div className="space-y-2">  
      <Label htmlFor="email">Email (optionnel)</Label>  
      <Input id="email" type="email" value={form.email} onChange={(e) => setForm({ ...form, email: e.ta  
    </div>  
    <Separator />  
  
    <div className="space-y-3">  
      <Label>Mode de récupération *</Label>  
      <RadioGroup value={deliveryMode} onChange={(v) => setDeliveryMode(v as DeliveryMode)} classNa  
      <label className={`flex items-center gap-3 p-4 border rounded-lg cursor-pointer transition-colors`  
        <RadioGroupItem value="livraison" id="livraison" />  
      </div>  
      <div className="flex items-center gap-2 font-medium text-sm">  
        <Truck className="w-4 h-4 text-primary" /> Livraison  
      </div>  
      <p className="text-xs text-muted-foreground">+{formatPrice(DELIVERY_FEE)}</p>  
    </div>  
    </label>  
    <div label className={`flex items-center gap-3 p-4 border rounded-lg cursor-pointer transition-colors`  
      <RadioGroupItem value="retrait" id="retrait" />  
    </div>  
    <div className="flex items-center gap-2 font-medium text-sm">  
      <Store className="w-4 h-4 text-accent" /> Retrait en magasin  
    </div>  
    <p className="text-xs text-muted-foreground">Gratuit</p>  
  </div>  
  </label>  
</RadioGroup>  
</div>  
  
{deliveryMode === 'livraison' &amp;&amp; (  
  <div className="space-y-2">
```

```

        <Label htmlFor="address">Adresse de livraison *</Label>
        <Input id="address" value={form.address} onChange={(e) => setForm({ ...form, address: e.target.value })}>
      </div>
    )}

    <div className="space-y-2">
      <Label htmlFor="notes">Notes (optionnel)</Label>
      <Input id="notes" value={form.notes} onChange={(e) => setForm({ ...form, notes: e.target.value })}>
    </div>
    </CardContent>
  </Card>
)}
</div>

{/* Right: summary */}
<div className="space-y-4">
  <Card className="border border-border sticky top-24">
    <CardHeader>
      <CardTitle>Récapitulatif</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
      {items.map((i) => (
        <div key={i.product.id} className="flex justify-between text-sm">
          <span className="text-muted-foreground truncate flex-1 min-w-0 pr-2">{i.product.name} × {i.quantity}</span>
          <span className="font-medium shrink-0">{formatPrice(i.quantity * i.product.price)}</span>
        </div>
      ))}
      <Separator />
      <div className="flex justify-between text-sm">
        <span className="text-muted-foreground">Sous-total</span>
        <span>{formatPrice(total)}</span>
      </div>
      <div className="flex justify-between text-sm">
        <span className="text-muted-foreground">Livraison</span>
        <span>{deliveryMode === 'retrait' ? 'Gratuit' : formatPrice(DELIVERY_FEE)}</span>
      </div>
      <Separator />
      <div className="flex justify-between font-bold text-lg">
        <span>Total</span>
        <span>{formatPrice(grandTotal)}</span>
      </div>

      {step === 'cart' ? (
        <Button size="lg" className="w-full bg-primary hover:bg-primary/90 text-primary-foreground mt-2" onClick={() => Passer la commande}>
          Passer la commande →
        </Button>
      ) : (
        <div className="space-y-2">
          <Button size="lg" className="w-full bg-primary hover:bg-primary/90 text-primary-foreground" onClick={() => {submitting ? 'Traitement...' : '✓ Confirmer la commande'}}>
            {submitting ? 'Traitement...' : '✓ Confirmer la commande'}
          </Button>
          <Button variant="ghost" size="sm" className="w-full" onClick={() => setStep('cart')}>
            ← Modifier le panier
          </Button>
        </div>
      )}
    </CardContent>
  </Card>
</div>
</div>
</div>
</div>
</MainLayout>
);
};

export default CartPage;

```

src/pages/ServicesPage.tsx

112 lignes | 5,987 octets

```
import React from 'react';
import { Link } from 'react-router-dom';
import { Zap, Sun, ChevronRight, Phone, FileText } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Card, CardContent } from '@components/ui/card';
import MainLayout from '@components/layouts/MainLayout';

const electricServices = [
  { title: 'Installation électrique', desc: 'Mise en place de nouveaux circuits, prises et interrupteurs selon les normes.' },
  { title: 'Dépannage', desc: 'Intervention rapide pour résoudre tout problème électrique urgent.' },
  { title: 'Branchement', desc: 'Raccordement au réseau SNEL et mise en service.' },
  { title: 'Rénovation', desc: 'Remise aux normes et modernisation de vos installations existantes.' },
  { title: 'Recherche de panne', desc: 'Diagnostic précis et localisation de défauts électriques.' },
  { title: 'Maintenance préventive', desc: 'Entretien régulier pour garantir la fiabilité de vos installations.' },
  { title: 'Tableau électrique', desc: 'Installation et mise à niveau de tableaux de distribution.' },
  { title: 'Éclairage', desc: 'Conception et installation de systèmes d'éclairage LED économiques.' },
];

const solarServices = [
  { title: 'Étude solaire', desc: 'Analyse de vos besoins et faisabilité d'un système solaire.' },
  { title: 'Dimensionnement', desc: 'Calcul précis de la puissance et capacité nécessaires.' },
  { title: 'Installation solaire', desc: 'Pose de panneaux, batteries et onduleurs par nos techniciens.' },
  { title: 'Maintenance solaire', desc: 'Entretien et nettoyage régulier pour optimiser le rendement.' },
  { title: 'Dépannage solaire', desc: 'Diagnostic et réparation de systèmes photovoltaïques.' },
  { title: 'Gestion des panneaux', desc: 'Monitoring et optimisation de la production solaire.' },
  { title: 'Batteries & stockage', desc: 'Installation de systèmes de stockage d'énergie.' },
  { title: 'Onduleurs', desc: 'Installation et configuration d'onduleurs pur sinus.' },
];

const ServicesPage: React.FC = () => {
  return (
    <MainLayout>
      { /* Header */ }
      <div className="hero-gradient border-b border-border">
        <div className="max-w-7xl mx-auto px-4 md:px-8 py-12">
          <h1 className="text-3xl md:text-4xl font-bold text-white mb-3">Nos Services</h1>
          <p className="text-secondary-foreground/70 text-lg">
            Expertise complète en électricité générale et solutions solaires
          </p>
        </div>
      </div>

      <div className="max-w-7xl mx-auto px-4 md:px-8 py-12 space-y-16">
        { /* Electrical services */ }
        <section>
          <div className="flex items-center gap-3 mb-8">
            <div className="w-12 h-12 bg-primary/10 rounded-lg flex items-center justify-center">
              <Zap className="w-6 h-6 text-primary" />
            </div>
            <div>
              <h2 className="text-2xl font-bold sca-heading-accent">Services Électriques</h2>
              <p className="text-muted-foreground text-sm mt-5">Installations, dépannage et maintenance</p>
            </div>
          </div>
          <div className="grid sm:grid-cols-2 md:grid-cols-4 gap-4">
            {electricServices.map((svc) => (
              <Card key={svc.title} className="border border-border hover:border-primary/40 transition-colors h-full">
                <CardContent className="p-5 h-full flex flex-col">
                  <ChevronRight className="w-4 h-4 text-primary mb-2" />
                  <h3 className="font-semibold text-foreground text-sm mb-2">{svc.title}</h3>
                  <p className="text-xs text-muted-foreground leading-relaxed flex-1">{svc.desc}</p>
                </CardContent>
              </Card>
            ))}
          </div>
        </section>
      </div>
    </MainLayout>
  );
};
```

```

        </div>
    </section>

    /* Solar services */
    <section>
        <div className="flex items-center gap-3 mb-8">
            <div className="w-12 h-12 bg-accent/10 rounded-lg flex items-center justify-center">
                <Sun className="w-6 h-6 text-accent" />
            </div>
            <div>
                <h2 className="text-2xl font-bold sca-heading-accent">Solutions Solaires</h2>
                <p className="text-muted-foreground text-sm mt-5">Énergie photovoltaïque pour votre indépendance</p>
            </div>
        </div>
        <div className="grid sm:grid-cols-2 md:grid-cols-4 gap-4">
            {solarServices.map((svc) => (
                <Card key={svc.title} className="border border-border hover:border-accent/40 transition-colors h-full">
                    <CardContent className="p-5 h-full flex flex-col">
                        <ChevronRight className="w-4 h-4 text-accent mb-2" />
                        <h3 className="font-semibold text-foreground text-sm mb-2">{svc.title}</h3>
                        <p className="text-xs text-muted-foreground leading-relaxed flex-1">{svc.desc}</p>
                    </CardContent>
                </Card>
            ))}
        </div>
    </section>

    /* CTA */
    <section className="bg-muted/40 rounded-xl p-8 text-center border border-border">
        <h2 className="text-2xl font-bold mb-3">Prêt à démarrer votre projet ?</h2>
        <p className="text-muted-foreground mb-6 max-w-xl mx-auto">
            Contactez SCA pour une intervention rapide ou demandez un devis gratuit pour vos travaux.
        </p>
        <div className="flex flex-wrap gap-4 justify-center">
            <Button asChild size="lg" className="bg-primary hover:bg-primary/90 text-primary-foreground gap-2">
                <Link to="/devis"><FileText className="w-4 h-4" />Demander un devis</Link>
            </Button>
            <Button asChild size="lg" variant="outline" className="gap-2">
                <a href="tel:+243858657475"><Phone className="w-4 h-4" />Appeler maintenant</a>
            </Button>
        </div>
    </section>
</div>
</MainLayout>
);
};

export default ServicesPage;

```

src/pages/DevisPage.tsx

198 lignes | 8,608 octets

```

import React, { useState } from 'react';
import { FileText, Upload, Loader2, CheckCircle } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Label } from '@components/ui/label';
import { Textarea } from '@components/ui/textarea';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select';
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from '@components/ui/card';
import MainLayout from '@components/layouts/MainLayout';
import { submitQuote } from '@services/api';
import { supabase } from '@db/supabase';
import { toast } from 'sonner';

const projectTypes = [

```

```

'Installation électrique résidentielle',
'Installation électrique commerciale',
'Rénovation électrique',
'Système solaire résidentiel',
'Système solaire commercial',
'Maintenance / Dépannage',
'Tableau électrique',
'Éclairage',
'Autre',
];

const toHex = (s: string) => {
  Array.from(new TextEncoder().encode(s), (b) => b.toString(16).padStart(2, '0')).join('');
};

const DevisPage: React.FC = () => {
  const [form, setForm] = useState({
    full_name: '', phone: '', email: '', project_type: '',
    description: '', address: ''
  });
  const [photos, setPhotos] = useState<File[]>([]);
  const [uploading, setUploading] = useState(false);
  const [submitted, setSubmitted] = useState(false);

  const handlePhotoChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const files = Array.from(e.target.files || []).slice(0, 5);
    setPhotos(files);
  };

  const uploadPhotos = async (): Promise<string[]> => {
    const urls: string[] = [];
    for (const file of photos) {
      const dot = file.name.lastIndexOf('.');
      const stem = dot > 0 ? file.name.slice(0, dot) : file.name;
      const ext = dot > 0 ? file.name.slice(dot) : '';
      const safeName = `${toHex(stem)}${ext}`;
      const path = `devis/${Date.now()}_${safeName}`;
      const { data, error } = await supabase.storage
        .from('quote-photos')
        .upload(path, file, { contentType: file.type });
      if (!error && data) {
        const { data: urlData } = supabase.storage.from('quote-photos').getPublicUrl(data.path);
        urls.push(urlData.publicUrl);
      }
    }
    return urls;
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!form.full_name.trim() || !form.phone.trim() || !form.project_type || !form.description.trim()) {
      toast.error('Veuillez remplir tous les champs obligatoires');
      return;
    }
    setUploading(true);
    try {
      const photoUrls = photos.length > 0 ? await uploadPhotos() : null;
      await submitQuote({
        full_name: form.full_name,
        phone: form.phone,
        email: form.email || null,
        project_type: form.project_type,
        description: form.description,
        address: form.address || null,
        photo_urls: photoUrls,
      });
      setSubmitted(true);
      toast.success('Votre demande de devis a été envoyée !');
    } catch (err) {

```

```

        console.error(err);
        toast.error('Erreur lors de l\'envoi. Veuillez réessayer.');
```

```

    } finally {
        setUploading(false);
    }
};
```

```

if (submitted) {
    return (
        <MainLayout>
        <div className="max-w-lg mx-auto px-4 py-20 text-center">
            <CheckCircle className="w-16 h-16 text-success mx-auto mb-4" />
            <h1 className="text-2xl font-bold mb-3">Demande envoyée !</h1>
            <p className="text-muted-foreground mb-8">
                Votre demande de devis a bien été reçue. L'équipe SCA vous contactera dans les 24h.
            </p>
            <Button onClick={() => { setSubmitted(false); setForm({ full_name: '', phone: '', email: '', project_type: '' }); }}>
                Faire une autre demande
            </Button>
        </div>
        </MainLayout>
    );
}
```

```

return (
    <MainLayout>
    <div className="hero-gradient border-b border-border">
        <div className="max-w-4xl mx-auto px-4 md:px-8 py-12">
            <h1 className="text-3xl font-bold text-white mb-3">Demander un Devis</h1>
            <p className="text-secondary-foreground/70">Décrivez votre projet, SCA vous répond sous 24h.</p>
        </div>
    </div>

    <div className="max-w-2xl mx-auto px-4 md:px-8 py-12">
        <Card className="border border-border">
            <CardHeader>
                <CardTitle className="flex items-center gap-2">
                    <FileText className="w-5 h-5 text-primary" />
                    Formulaire de devis
                </CardTitle>
                <CardDescription>Les champs marqués * sont obligatoires</CardDescription>
            </CardHeader>
            <CardContent>
                <form onSubmit={handleSubmit} className="space-y-5">
                    <div className="grid sm:grid-cols-2 gap-4">
                        <div className="space-y-2">
                            <Label htmlFor="full_name">Nom complet *</Label>
                            <Input id="full_name" value={form.full_name} onChange={(e) => { setForm({ ...form, full_name: e.target.value }); }} />
                        </div>
                        <div className="space-y-2">
                            <Label htmlFor="phone">Téléphone *</Label>
                            <Input id="phone" value={form.phone} onChange={(e) => { setForm({ ...form, phone: e.target.value }); }} />
                        </div>
                    </div>

                    <div className="space-y-2">
                        <Label htmlFor="email">Email (optionnel)</Label>
                        <Input id="email" type="email" value={form.email} onChange={(e) => { setForm({ ...form, email: e.target.value }); }} />
                    </div>

                    <div className="space-y-2">
                        <Label>Type de projet *</Label>
                        <Select value={form.project_type} onChange={(v) => { setForm({ ...form, project_type: v }); }}>
                            <SelectTrigger>
                                <SelectValue placeholder="Choisir un type de projet" />
                            </SelectTrigger>
                            <SelectContent>
                                {projectTypes.map((t) => (
                                    <SelectItem key={t} value={t}>{t}</SelectItem>
                                ))}
                            </SelectContent>
                        </Select>
                    </div>
                </form>
            </CardContent>
        </Card>
    </div>
</div>
</MainLayout>
);
```

```

        </SelectContent>
      </Select>
    </div>

    <div className="space-y-2">
      <Label htmlFor="description">Description du projet *</Label>
      <Textarea
        id="description"
        value={form.description}
        onChange={(e) => setForm({ ...form, description: e.target.value })}
        placeholder="Décrivez votre projet en détail : type de travaux, superficie, équipements souhaités..."
        rows={5}
        required
      />
    </div>

    <div className="space-y-2">
      <Label htmlFor="address">Adresse / Zone</Label>
      <Input id="address" value={form.address} onChange={(e) => setForm({ ...form, address: e.target.value }} />
    </div>

    <div className="space-y-2">
      <Label htmlFor="photos">Photos du projet (optionnel, max 5)</Label>
      <div className="border border-dashed border-border rounded-lg p-4 text-center cursor-pointer hover:border-2">
        <input id="photos" type="file" multiple accept="image/*" onChange={handlePhotoChange} className="hidden" />
        <label htmlFor="photos" className="cursor-pointer">
          <Upload className="w-8 h-8 text-muted-foreground mx-auto mb-2" />
          <p className="text-sm text-muted-foreground">
            {photos.length > 0
              ? `${photos.length} photo${photos.length > 1 ? 's' : ''} sélectionnée${photos.length > 1 ? 's' : ''}
              : 'Cliquez pour ajouter des photos'}
          </p>
        </label>
      </div>
    </div>

    <Button type="submit" size="lg" className="w-full bg-primary hover:bg-primary/90 text-primary-foreground gap-2">
      {uploading ? <Loader2 className="w-4 h-4 animate-spin" /> : <img alt="arrow" />} Envoi en cours...
    </Button>
  </form>
</CardContent>
</Card>
</div>
</MainLayout>
);
};

export default DevisPage;

```

src/pages/ContactPage.tsx

185 lignes | 9,238 octets

```

import React, { useState } from 'react';
import { Phone, Mail, MapPin, MessageCircle, Lightbulb, CheckCircle, Loader2 } from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Textarea } from '@/components/ui/textarea';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import MainLayout from '@/components/layouts/MainLayout';
import { submitStudyRequest } from '@/services/api';
import { toast } from 'sonner';

const ContactPage: React.FC = () => {
  const [studyForm, setStudyForm] = useState({ full_name: '', phone: '', email: '', need_description: '' });

```

```

const [studyDone, setStudyDone] = useState(false);
const [loading, setLoading] = useState(false);

const handleStudy = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!studyForm.full_name.trim() || !studyForm.phone.trim() || !studyForm.need_description.trim()) {
    toast.error('Veuillez remplir les champs obligatoires');
    return;
  }
  setLoading(true);
  try {
    await submitStudyRequest({
      full_name: studyForm.full_name,
      phone: studyForm.phone,
      email: studyForm.email || null,
      need_description: studyForm.need_description,
    });
    setStudyDone(true);
    toast.success('Demande d\'étude envoyée !');
  } catch (err) {
    console.error(err);
    toast.error('Erreur lors de l\'envoi. Réessayez.');
```

} finally {
 setLoading(false);
 }
 }
};

return (
 <MainLayout>
 </* Header */>
 <div className="hero-gradient border-b border-border">
 <div className="max-w-7xl mx-auto px-4 md:px-8 py-12">
 <h1 className="text-3xl md:text-4xl font-bold text-white mb-3">Contact</h1>
 <p className="text-secondary-foreground/70 text-lg">Nous sommes disponibles pour vous aider</p>
 </div>
 </div>

 <div className="max-w-7xl mx-auto px-4 md:px-8 py-12 grid md:grid-cols-2 gap-10">
 </* Left - contact info + map */>
 <div className="space-y-6">
 <div>
 <h2 className="text-xl font-bold sca-heading-accent mb-6">Nos coordonnées</h2>
 </div>

 <div className="grid sm:grid-cols-2 gap-4">
 <[
 { icon: MapPin, label: 'Adresse', value: 'Haut-Katanga, République Démocratique du Congo', href: undefined, color: 'text-accent' },
 { icon: Phone, label: 'Téléphone 1', value: '085 865 74 75', href: 'tel:+243858657475', color: 'text-accent' },
 { icon: Phone, label: 'Téléphone 2', value: '097 528 31 55', href: 'tel:+243975283155', color: 'text-accent' },
 { icon: MessageCircle, label: 'WhatsApp', value: 'Discuter sur WhatsApp', href: 'https://wa.me/243858657475', color: 'text-accent' },
 { icon: Mail, label: 'Email', value: 'contact@sca-katanga.com', href: 'mailto:contact@sca-katanga.com', color: 'text-accent' }
].map((item) => (
 <Card key={item.label} className="border border-border">
 <CardContent className="p-4 flex items-start gap-3">
 <item.icon className={`w-5 h-5 mt-0.5 shrink-0 \${item.color}`} />
 </div>
 <p className="text-xs text-muted-foreground font-medium uppercase tracking-wide">{item.label}</p>
 {item.href ? (
 <a href={item.href} target={item.href.startsWith('http') ? '_blank' : undefined} rel="noreferrer"
 className="text-sm font-medium text-foreground hover:text-primary transition-colors">
 {item.value}

) : (
 <p className="text-sm font-medium text-foreground">{item.value}</p>
)
)
 </CardContent>
 </Card>
 </div>
 </div>
 </div>
 </MainLayout>
);

```

    )})
</div>

{/* Quick contact buttons */}
<div className="flex flex-wrap gap-3 pt-2">
  <Button asChild className="bg-primary hover:bg-primary/90 text-primary-foreground gap-2">
    <a href="tel:+243858657475"><Phone className="w-4 h-4" /><Appeler SCA</a>
  </Button>
  <Button asChild variant="outline" className="gap-2">
    <a href="https://wa.me/243858657475" target="_blank" rel="noreferrer">
      <MessageCircle className="w-4 h-4 text-success" /><WhatsApp
    </a>
  </Button>
</div>

{/* Google Maps embed */}
<div className="rounded-lg overflow-hidden border border-border shadow-sm">
  <iframe
    width="100%"
    height="280"
    frameBorder="0"
    style={{ border: 0 }}
    referrerPolicy="no-referrer-when-downgrade"
    src="https://www.google.com/maps/embed/v1/place?key=AIZAyB_LJOYJL-84SMuxNB7LtRGhxEQLjswvy0&q=Haut-Katanga"
    allowFullScreen
    title="Localisation SCA - Haut-Katanga, RDC"
  />
</div>
</div>

{/* Right - Study request form */}
<div>
  <Card className="border border-border">
    <CardHeader>
      <div className="flex items-center gap-3">
        <div className="w-10 h-10 bg-accent/10 rounded-lg flex items-center justify-center">
          <Lightbulb className="w-5 h-5 text-accent" />
        </div>
        <div>
          <CardTitle>Je ne sais pas quoi acheter</CardTitle>
          <p className="text-sm text-muted-foreground mt-1">Décrivez votre besoin, SCA vous conseille gratuitement
        </p>
        </div>
      </div>
    </CardHeader>
    <CardContent>
      {studyDone ? (
        <div className="text-center py-8">
          <CheckCircle className="w-14 h-14 text-success mx-auto mb-4" />
          <p className="font-semibold text-foreground text-lg mb-2">Demande reçue !</p>
          <p className="text-muted-foreground text-sm mb-6">
            Notre équipe analyse votre besoin et vous contacte sous 24h avec une recommandation personnalisée.
          </p>
          <Button variant="outline" onClick={() => { setStudyDone(false); setStudyForm({ full_name: '', phone: '' }); }}>
            Faire une autre demande
          </Button>
        </div>
      ) : (
        <form onSubmit={handleStudy} className="space-y-4">
          <div className="bg-accent/5 border border-accent/20 rounded-lg p-3 text-sm text-muted-foreground">
            <strong>Exemple :</strong> « Je veux alimenter une pompe de 1,5
          </div>

          <div className="grid sm:grid-cols-2 gap-4">
            <div className="space-y-2">
              <Label htmlFor="study_name">Nom complet </Label>
              <Input id="study_name" value={studyForm.full_name} onChange={(e) => { setStudyForm({ ...studyForm, full_name: e.target.value }} }} />
            </div>
            <div className="space-y-2">

```

```

        <Label htmlFor="study_phone">Téléphone *</Label>
        <Input id="study_phone" value={studyForm.phone} onChange={(e) => setStudyForm({ ...studyForm, phone: e.target.value })}>
      </div>
    </div>

    <div className="space-y-2">
      <Label htmlFor="study_email">Email (optionnel)</Label>
      <Input id="study_email" type="email" value={studyForm.email} onChange={(e) => setStudyForm({ ...studyForm, email: e.target.value })}>
    </div>

    <div className="space-y-2">
      <Label htmlFor="study_desc">Décrivez votre besoin *</Label>
      <Textarea
        id="study_desc"
        value={studyForm.need_description}
        onChange={(e) => setStudyForm({ ...studyForm, need_description: e.target.value })}
        placeholder="Ex : Je veux alimenter ma maison avec l'énergie solaire, surface 80m², 4 pièces..."
        rows={5}
        required
      />
    </div>

    <Button type="submit" size="lg" className="w-full bg-accent hover:bg-accent/90 text-accent-foreground" disabled={loading} >
      <Loader2 className="w-4 h-4 animate-spin" /> Envoyer
    </Button>
  </form>
)
</CardContent>
</Card>
</div>
</div>
</MainLayout>
);
};

export default ContactPage;

```

src/pages/ConseilPage.tsx

146 lignes | 6,831 octets

```

import React, { useState } from 'react';
import { Lightbulb, CheckCircle, Loader2 } from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Textarea } from '@/components/ui/textarea';
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from '@/components/ui/card';
import MainLayout from '@/components/layouts/MainLayout';
import { submitStudyRequest } from '@/services/api';
import { toast } from 'sonner';

const ConseilPage: React.FC = () => {
  const [form, setForm] = useState({ full_name: '', phone: '', email: '', need_description: '' });
  const [done, setDone] = useState(false);
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!form.full_name.trim() || !form.phone.trim() || !form.need_description.trim()) {
      toast.error('Veuillez remplir les champs obligatoires');
      return;
    }
    setLoading(true);
    try {
      await submitStudyRequest({

```

```

        full_name: form.full_name,
        phone: form.phone,
        email: form.email || null,
        need_description: form.need_description,
    });
    setDone(true);
    toast.success('Demande d\'étude envoyée !');
} catch (err) {
    console.error(err);
    toast.error('Erreur lors de l\'envoi.');
```

```

    } finally {
        setLoading(false);
    }
}

};

const examples = [
    'Je veux alimenter une pompe de 1,5 HP avec du solaire.',
    'J\'ai une boutique de 50m² à éclairer entièrement en LED.',
    'Je souhaite une autonomie de 3 jours pour ma villa.',
    'Je veux installer 10 prises et 5 interrupteurs dans ma maison neuve.',
];

return (
    <MainLayout>
        <div className="hero-gradient border-b border-border">
            <div className="max-w-4xl mx-auto px-4 md:px-8 py-12">
                <div className="flex items-center gap-3 mb-3">
                    <Lightbulb className="w-8 h-8 text-accent" />
                    <h1 className="text-3xl md:text-4xl font-bold text-white">Conseil / Étude gratuite</h1>
                </div>
                <p className="text-secondary-foreground/70 text-lg">
                    Vous ne savez pas quoi acheter ? Décrivez votre besoin, nos experts SCA vous guident.
                </p>
            </div>
            <div className="max-w-4xl mx-auto px-4 md:px-8 py-12 grid md:grid-cols-5 gap-8">
                { /* Left: examples */ }
                <div className="md:col-span-2 space-y-4">
                    <h2 className="font-bold text-foreground text-lg">Exemples de demandes</h2>
                    <div className="space-y-3">
                        {examples.map((ex) => (
                            <div key={ex} className="bg-muted/50 border border-border rounded-lg p-3 text-sm text-muted-foreground italic">
                                onClick={() => setForm((f) => ({ ...f, need_description: ex })))}
                                « {ex} »
                            </div>
                        ))}
                    </div>
                    <p className="text-xs text-muted-foreground">Cliquez sur un exemple pour le copier</p>
                </div>
                <div className="bg-accent/5 border border-accent/20 rounded-lg p-4 mt-6">
                    <p className="text-sm font-semibold text-foreground mb-2">Pourquoi demander une étude ?</p>
                    <ul className="text-sm text-muted-foreground space-y-1.5">
                        <li>■ Évitez les mauvais achats</li>
                        <li>■ Dimensionnement précis</li>
                        <li>■ Conseil d'expert gratuit</li>
                        <li>■ Réponse sous 24h</li>
                    </ul>
                </div>
            </div>
            { /* Right: form */ }
            <div className="md:col-span-3">
                <Card className="border border-border">
                    <CardHeader>
                        <CardTitle>Décrivez votre besoin</CardTitle>
                        <CardDescription>Les champs * sont obligatoires. Réponse sous 24h.</CardDescription>
                    </CardHeader>

```

```

    <CardContent>
      {done ? (
        <div className="text-center py-10">
          <CheckCircle className="w-16 h-16 text-success mx-auto mb-4" />
          <p className="font-bold text-xl mb-2">Demande reçue !</p>
          <p className="text-muted-foreground text-sm mb-6 max-w-sm mx-auto">
            Nos experts analysent votre besoin et vous contactent très prochainement avec une recommandation personnalisée.
          </p>
          <Button onClick={() => { setDone(false); setForm({ full_name: '', phone: '', email: '', need_description: '' }); }}>
            Nouvelle demande
          </Button>
        </div>
      ) : (
        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="grid sm:grid-cols-2 gap-4">
            <div className="space-y-2">
              <Label htmlFor="cn">Nom complet </Label>
              <Input id="cn" value={form.full_name} onChange={(e) => { setForm({ ...form, full_name: e.target.value }); }} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="cp">Téléphone </Label>
              <Input id="cp" value={form.phone} onChange={(e) => { setForm({ ...form, phone: e.target.value }); }} />
            </div>
          </div>
          <div className="space-y-2">
            <Label htmlFor="ce">Email (optionnel)</Label>
            <Input id="ce" type="email" value={form.email} onChange={(e) => { setForm({ ...form, email: e.target.value }); }} />
          </div>
          <div className="space-y-2">
            <Label htmlFor="cd">Décrivez votre besoin </Label>
            <Textarea
              id="cd"
              value={form.need_description}
              onChange={(e) => { setForm({ ...form, need_description: e.target.value }); }}
              placeholder="Soyez précis : type d'installation, superficie, appareils à alimenter, budget estimé..."
              rows={6}
              required
            />
          </div>
          <Button type="submit" size="lg" className="w-full bg-primary hover:bg-primary/90 text-primary-foreground">
            {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : Envoyer
          </Button>
        </form>
      )}
    </CardContent>
  </Card>
</div>
</div>
</MainLayout>
);
};

export default ConseilPage;

```

● Espace Administrateur

src/pages/admin/AdminLoginPage.tsx

111 lignes | 4,640 octets

```

import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';

```

```

import { Zap, Eye, EyeOff, Loader2 } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Label } from '@components/ui/label';
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from '@components/ui/card';
import { supabase } from '@db/supabase';
import { toast } from 'sonner';

const AdminLoginPage: React.FC = () => {
  const navigate = useNavigate();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPw, setShowPw] = useState(false);
  const [loading, setLoading] = useState(false);

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!email.trim() || !password.trim()) { toast.error('Remplissez tous les champs'); return; }
    setLoading(true);
    try {
      const { data, error } = await supabase.auth.signInWithPassword({ email, password });
      if (error) { toast.error('Identifiants incorrects'); return; }
      // Check admin role
      const { data: profile } = await supabase.from('profiles').select('role').eq('id', data.user.id).maybeSingle();
      if (profile?.role !== 'admin') {
        await supabase.auth.signOut();
        toast.error('Accès réservé aux administrateurs');
        return;
      }
      toast.success('Connexion réussie');
      navigate('/admin/dashboard');
    } catch (err) {
      console.error(err);
      toast.error('Erreur de connexion');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-secondary flex items-center justify-center p-4">
      <div className="w-full max-w-md">
        <div className="flex flex-col items-center justify-center">
          <div className="flex flex-col items-center justify-center mb-8">
            
            <h1 className="text-2xl font-bold text-white">Espace Administrateur</h1>
            <p className="text-secondary-foreground/60 text-sm mt-1">SCA – Société du Courant Alternatif</p>
          </div>
          <div>
            <Card className="border border-border/40">
              <CardHeader>
                <CardTitle className="flex items-center gap-2">
                  <Zap className="w-5 h-5 text-primary" />
                  Connexion Admin
                </CardTitle>
                <CardDescription>Accès réservé au personnel SCA autorisé</CardDescription>
              </CardHeader>
              <CardContent>
                <form onSubmit={handleLogin} className="space-y-4">
                  <div className="space-y-2">
                    <Label htmlFor="admin-email">Email</Label>
                    <Input
                      id="admin-email"
                      type="email"

```

```

        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="admin@sca.com"
        autoComplete="email"
        required
      />
    </div>
    <div className="space-y-2">
      <Label htmlFor="admin-pw">Mot de passe</Label>
      <div className="relative">
        <Input
          id="admin-pw"
          type={showPw ? 'text' : 'password'}
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="....."
          autoComplete="current-password"
          required
          className="pr-10"
        />
        <button type="button" className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground hover:
          {showPw ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
        </button>
      </div>
    </div>
    <Button type="submit" size="lg" className="w-full bg-primary hover:bg-primary/90 text-primary-foreground"
      {loading ? <&Loader2 className="w-4 h-4 animate-spin mr-2" />Connexion...& : 'Se connect
    </Button>
  </form>
  </CardContent>
  </Card>

  <p className="text-center text-xs text-secondary-foreground/40 mt-6">
    © {new Date().getFullYear()} SCA - Administration
  </p>
</div>
</div>
);
};

export default AdminLoginPage;

```

src/pages/admin/AdminLayout.tsx

156 lignes | 5,731 octets

```

import React, { useState, useEffect } from 'react';
import { Link, useLocation, useNavigate } from 'react-router-dom';
import {
  LayoutDashboard, Package, ShoppingBag, FileText, MessageSquare,
  Users, BarChart2, Menu, X, Zap, Logout, ChevronRight
} from 'lucide-react';
import { Button } from '@components/ui/button';
import { Sheet, SheetContent } from '@components/ui/sheet';
import { supabase } from '@db/supabase';
import { toast } from 'sonner';

const navItems = [
  { label: 'Tableau de bord', path: '/admin/dashboard', icon: LayoutDashboard },
  { label: 'Produits', path: '/admin/produits', icon: Package },
  { label: 'Stock', path: '/admin/stock', icon: BarChart2 },
  { label: 'Commandes', path: '/admin/commandes', icon: ShoppingBag },
  { label: 'Factures', path: '/admin/factures', icon: FileText },
  { label: 'Devis', path: '/admin/devis', icon: MessageSquare },
  { label: 'Clients', path: '/admin/clients', icon: Users },
];

```

```

const AdminNavItem: React.FC<{ item: typeof navItems[0]; active: boolean; onClick?: () => void }> = ({ item, active
  <Link
    to={item.path}
    onClick={onClick}
    className={`flex items-center gap-3 px-3 py-2.5 rounded-lg text-sm font-medium transition-colors ${
      active
        ? 'bg-sidebar-primary text-sidebar-primary-foreground'
        : 'text-sidebar-foreground/70 hover:bg-sidebar-accent hover:text-sidebar-foreground'
    }}
  >
    <item.icon className="w-4 h-4 shrink-0" />
    <span className="truncate">{item.label}</span>
    {active & & <ChevronRight className="w-3.5 h-3.5 ml-auto shrink-0" />}
  </Link>
);

const AdminSidebarContent: React.FC<{ onNav?: () => void }> = ({ onNav }) => {
  const location = useLocation();
  const navigate = useNavigate();

  const handleLogout = async () => {
    await supabase.auth.signOut();
    toast.success('Déconnecté');
    navigate('/admin');
  };

  return (
    <div className="flex flex-col h-full bg-sidebar">
      </* Logo */>
      <div className="flex items-center gap-3 px-4 py-5 border-b border-sidebar-border">
        
        <div>
          <p className="font-bold text-sidebar-foreground text-sm">SCA Admin</p>
          <p className="text-xs text-sidebar-foreground/50">Gestion</p>
        </div>
      </div>

      </* Nav */>
      <nav className="flex-1 px-3 py-4 space-y-1 overflow-y-auto">
        {navItems.map((item) => (
          <AdminNavItem
            key={item.path}
            item={item}
            active={location.pathname === item.path}
            onClick={onNav}
          />
        ))}
      </nav>

      </* Footer */>
      <div className="px-3 py-4 border-t border-sidebar-border space-y-2">
        <Link
          to="/"
          className="flex items-center gap-3 px-3 py-2 rounded-lg text-sm text-sidebar-foreground/60 hover:bg-sidebar-accent
        >
          <Zap className="w-4 h-4" />
          Voir le site
        </Link>
        <button
          onClick={handleLogout}
          className="flex items-center gap-3 px-3 py-2 rounded-lg text-sm text-destructive/80 hover:bg-destructive/10 trans
        >
          <Logout className="w-4 h-4" />
          Se déconnecter
      </div>
    </div>
  );
};

```

```

        </button>
    </div>
</div>
);
};

const AdminLayout: React.FC<{ children: React.ReactNode }> = ({ children }) => {
    const [mobileOpen, setMobileOpen] = useState(false);
    const location = useLocation();
    const navigate = useNavigate();
    const [checking, setChecking] = useState(true);

    useEffect(() => {
        supabase.auth.getSession().then(async ({ data }) => {
            if (!data.session) { navigate('/admin'); return; }
            const { data: profile } = await supabase.from('profiles').select('role').eq('id', data.session.user.id).maybeSingle()
            if (profile?.role !== 'admin') { await supabase.auth.signOut(); navigate('/admin'); return; }
            setChecking(false);
        });
    }, [navigate]);

    // Close mobile sidebar on route change
    useEffect(() => { setMobileOpen(false); }, [location.pathname]);

    if (checking) {
        return (
            <div className="flex items-center justify-center min-h-screen bg-background">
                <div className="animate-spin w-8 h-8 border-2 border-primary border-t-transparent rounded-full" />
            </div>
        );
    }

    return (
        <div className="flex min-h-screen w-full bg-background">
            {/* Desktop sidebar */}
            <aside className="hidden md:flex flex-col w-64 shrink-0 border-r border-border">
                <AdminSidebarContent />
            </aside>

            {/* Mobile sidebar */}
            <Sheet open={mobileOpen} onOpenChange={setMobileOpen}>
                <SheetContent side="left" className="p-0 w-64">
                    <AdminSidebarContent onNav={() => setMobileOpen(false)} />
                </SheetContent>
            </Sheet>

            {/* Main content */}
            <div className="flex-1 min-w-0 flex flex-col">
                {/* Mobile header */}
                <header className="md:hidden flex items-center gap-3 px-4 py-3 border-b border-border bg-card">
                    <Button variant="ghost" size="icon" onClick={() => setMobileOpen(true)}>
                        <Menu className="w-5 h-5" />
                    </Button>
                    <span className="font-semibold text-sm text-foreground">
                        {navItems.find((n) => n.path === location.pathname)?.label || 'Administration'}
                    </span>
                </header>

                <main className="flex-1 overflow-x-hidden p-4 md:p-6">
                    {children}
                </main>
            </div>
        </div>
    );
};

export default AdminLayout;

```

src/pages/admin/AdminDashboard.tsx

154 lignes | 6,868 octets

```
import React, { useEffect, useState } from 'react';
import { Package, ShoppingBag, MessageSquare, Users, TrendingUp, AlertTriangle, XCircle } from 'lucide-react';
import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Cell } from 'recharts';
import AdminLayout from '../AdminLayout';
import { fetchDashboardStats } from '@services/api';
import { formatPrice } from '@lib/helpers';

interface Stats {
  todaySales: number;
  totalOrders: number;
  pendingQuotes: number;
  totalClients: number;
  totalProducts: number;
  enStock: number;
  stockFaible: number;
  rupture: number;
}

const StatCard: React.FC<{ label: string; value: string | number; icon: React.ElementType; color: string; sub?: string }> = ({
  label, value, icon: Icon, color, sub,
}) => {
  <Card className="border border-border">
    <CardContent className="p-5">
      <div className="flex items-start justify-between">
        <div>
          <p className="text-sm text-muted-foreground">{label}</p>
          <p className={`text-2xl font-bold mt-1 ${color}`}>{value}</p>
          {sub & & <p className="text-xs text-muted-foreground mt-0.5">{sub}</p>}
        </div>
        <div className={`w-10 h-10 rounded-lg flex items-center justify-center ${color.replace('text-', 'bg-')}/10`>
          <Icon className={`w-5 h-5 ${color}`} />
        </div>
      </div>
    </CardContent>
  </Card>
};

const AdminDashboard: React.FC = () => {
  const [stats, setStats] = useState<Stats | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchDashboardStats()
      .then((s) => setStats(s as Stats))
      .catch(console.error)
      .finally(() => setLoading(false));
  }, []);

  const stockChartData = stats
    ? [
        { name: 'En stock', value: stats.enStock, fill: 'hsl(var(--success))' },
        { name: 'Stock faible', value: stats.stockFaible, fill: 'hsl(var(--warning))' },
        { name: 'Rupture', value: stats.rupture, fill: 'hsl(var(--destructive))' },
      ]
    : [];

  return (
    <AdminLayout>
      <div className="space-y-6">
        <div>
          <h1 className="text-2xl font-bold text-foreground">Tableau de bord</h1>
          <p className="text-muted-foreground text-sm mt-1">
            {new Date().toLocaleDateString('fr-FR', { weekday: 'long', year: 'numeric', month: 'long', day: 'numeric' })}
          </p>
        </div>
      </div>
    </AdminLayout>
  );
};
```

```
<div></p>>
</div>>

{loading ? (
  <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
    [...Array(8)].map((_, i) => {
      <div key={i} className="h-24 bg-muted animate-pulse rounded-lg" />
    })
  </div>
) : stats ? (
  <&gt;
    { /* KPI Cards */ }
    <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
      <StatCard label="Ventes du jour" value={formatPrice(stats.todaySales)} icon={TrendingUp} color="text-success"/>
      <StatCard label="Total commandes" value={stats.totalOrders} icon={ShoppingBag} color="text-primary" />
      <StatCard label="Devis en attente" value={stats.pendingQuotes} icon={MessageSquare} color="text-accent" />
      <StatCard label="Clients" value={stats.totalClients} icon={Users} color="text-foreground" />
    </div>

    { /* Stock cards */ }
    <div>
      <h2 className="text-lg font-semibold mb-3">État du stock ({stats.totalProducts} produits)</h2>
      <div className="grid grid-cols-3 gap-4">
        <Card className="border border-border bg-green-500/5">
          <CardContent className="p-4 flex items-center gap-3">
            <div className="w-8 h-8 bg-green-500/10 rounded-full flex items-center justify-center">
              <Package className="w-4 h-4 text-green-500" />
            </div>
            <p className="text-xs text-muted-foreground">En stock</p>
            <p className="text-xl font-bold text-green-500">{{stats.enStock}}</p>
          </CardContent>
        </Card>
        <Card className="border border-border bg-yellow-500/5">
          <CardContent className="p-4 flex items-center gap-3">
            <div className="w-8 h-8 bg-yellow-500/10 rounded-full flex items-center justify-center">
              <AlertTriangle className="w-4 h-4 text-yellow-500" />
            </div>
            <p className="text-xs text-muted-foreground">Stock faible</p>
            <p className="text-xl font-bold text-yellow-500">{{stats.stockFaible}}</p>
          </CardContent>
        </Card>
        <Card className="border border-border bg-destructive/5">
          <CardContent className="p-4 flex items-center gap-3">
            <div className="w-8 h-8 bg-destructive/10 rounded-full flex items-center justify-center">
              <XCircle className="w-4 h-4 text-destructive" />
            </div>
            <p className="text-xs text-muted-foreground">Rupture</p>
            <p className="text-xl font-bold text-destructive">{{stats.rupture}}</p>
          </CardContent>
        </Card>
      </div>

      { /* Stock chart */ }
      <Card className="border border-border">
        <CardHeader>
          <CardTitle className="text-base">Répartition du stock</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="w-full min-w-0 overflow-hidden" style={{ height: 200 }}>
            <ResponsiveContainer width="100%" height="100%">
              <BarChart data={stockChartData} barSize={48}>
                <CartesianGrid strokeDasharray="3 3" className="opacity-30" />
                <XAxis dataKey="name" tick={{ fontSize: 12 }} />
```

```

        <YAxis tick={{ fontSize: 12 }} />
        <Tooltip formatter={(v) => [v, 'Produits']} />
        <Bar dataKey="value" radius={[4, 4, 0, 0]}>
          {stockChartData.map((entry, index) => (
            <Cell key={`cell-${index}`} fill={entry.fill} />
          ))}
        </Bar>
      </BarChart>
    </ResponsiveContainer>
  </div>
  </CardContent>
</Card>
</>
) : (
  <p className="text-muted-foreground">Impossible de charger les statistiques.</p>
)
</div>
</AdminLayout>
);
};

export default AdminDashboard;

```

src/pages/admin/AdminProducts.tsx

257 lignes | 12,342 octets

```

import React, { useEffect, useState, useCallback } from 'react';
import { Plus, Pencil, Trash2, Search, Package, Loader2, X } from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Textarea } from '@/components/ui/textarea';
import { Badge } from '@/components/ui/badge';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@/components/ui/select';
import {
  Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter
} from '@/components/ui/dialog';
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
  AlertDialogDescription, AlertDialogFooter, AlertDialogHeader, AlertDialogTitle
} from '@/components/ui/alert-dialog';
import AdminLayout from '@/AdminLayout';
import { fetchAllProducts, fetchCategories, upsertProduct, deleteProduct } from '@/services/api';
import { getStockStatus, stockLabel, stockBadgeClass, formatPrice } from '@/lib/helpers';
import type { Product, Category } from '@/types/index';
import { toast } from 'sonner';

const emptyForm = {
  name: '', reference: '', description: '', category_id: '',
  price: '', stock_quantity: '', stock_threshold: '', image_url: '', is_active: true,
};

const AdminProducts: React.FC = () => {
  const [products, setProducts] = useState<Product[]>([]);
  const [categories, setCategories] = useState<Category[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [dialogOpen, setDialogOpen] = useState(false);
  const [deleteId, setDeleteId] = useState<string | null>(null);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [form, setForm] = useState(emptyForm);
  const [saving, setSaving] = useState(false);

  const load = useCallback(() => {
    setLoading(true);
  }, []);

```

```

    Promise.all([fetchAllProducts(), fetchCategories()])
      .then(([prods, cats]) => { setProducts(prods); setCategories(cats); })
      .catch(console.error)
      .finally(() => { setLoading(false); });
}, []);

useEffect(() => { load(); }, [load]);

const filtered = products.filter((p) => {
  !search.trim() ||
  p.name.toLowerCase().includes(search.toLowerCase()) ||
  p.reference.toLowerCase().includes(search.toLowerCase())
});

const openCreate = () => {
  setEditingId(null);
  setForm(emptyForm);
  setDialogOpen(true);
};

const openEdit = (p: Product) => {
  setEditingId(p.id);
  setForm({
    name: p.name, reference: p.reference, description: p.description || '',
    category_id: p.category_id, price: String(p.price),
    stock_quantity: String(p.stock_quantity), stock_threshold: String(p.stock_threshold),
    image_url: p.image_url || '', is_active: p.is_active,
  });
  setDialogOpen(true);
};

const handleSave = async () => {
  if (!form.name.trim() || !form.reference.trim() || !form.category_id || !form.price) {
    toast.error('Remplissez les champs obligatoires'); return;
  }
  setSaving(true);
  try {
    await upsertProduct({
      ...(editingId ? { id: editingId } : {}),
      name: form.name.trim(),
      reference: form.reference.trim(),
      description: form.description.trim() || null,
      category_id: form.category_id,
      price: parseFloat(form.price),
      stock_quantity: parseInt(form.stock_quantity) || 0,
      stock_threshold: parseInt(form.stock_threshold) || 5,
      image_url: form.image_url.trim() || null,
      is_active: form.is_active,
    });
    toast.success(editingId ? 'Produit mis à jour' : 'Produit créé');
    setDialogOpen(false);
    load();
  } catch (err) {
    console.error(err);
    toast.error('Erreur lors de l\'enregistrement');
  } finally {
    setSaving(false);
  }
};

const handleDelete = async () => {
  if (!deleteId) return;
  try {
    await deleteProduct(deleteId);
    toast.success('Produit supprimé');
    load();
  } catch (err) {
    console.error(err);
  }
};

```

```

        toast.error('Erreur lors de la suppression');
    } finally {
        setDeleteId(null);
    }
};

return (
    <AdminLayout>
    <div className="space-y-5">
        <div className="flex items-center justify-between gap-3 flex-wrap">
            <div>
                <h1 className="text-2xl font-bold">Gestion des produits</h1>
                <p className="text-muted-foreground text-sm">{products.length} produits au total</p>
            </div>
            <Button onClick={openCreate} className="bg-primary hover:bg-primary/90 text-primary-foreground gap-2 shrink-0">
                <Plus className="w-4 h-4" /> Nouveau produit
            </Button>
        </div>

        {/* Search */}
        <div className="relative max-w-sm">
            <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
            <Input placeholder="Rechercher..." value={search} onChange={(e) => setSearch(e.target.value)} className="pl-10" />
            <button onClick={() => setSearch('')} className="absolute right-3 top-1/2 -translate-y-1/2" />
        </div>

        {/* Table */}
        <div className="w-full max-w-full overflow-x-auto bg-card rounded-lg border border-border">
            <table className="w-full min-w-max text-sm">
                <thead>
                    <tr className="border-b border-border">
                        {[ 'Réf', 'Produit', 'Catégorie', 'Prix', 'Stock', 'État', 'Actions' ].map((h) => (
                            <th key={h} className="whitespace-nowrap text-left px-4 py-3 text-xs font-semibold text-muted-foreground">{h}</th>
                        ))}
                    </tr>
                </thead>
                <tbody>
                    {loading ? (
                        <tr>
                            <td colspan={7} className="text-center py-12 text-muted-foreground"><Loader2 className="w-8 h-8" /></td>
                        </tr>
                    ) : filtered.length === 0 ? (
                        <tr>
                            <td colspan={7} className="text-center py-12 text-muted-foreground">Aucun produit trouvé</td>
                        </tr>
                    ) : filtered.map((p) => {
                        const status = getStockStatus(p);
                        return (
                            <tr key={p.id} className="border-b border-border/50 hover:bg-muted/30 transition-colors">
                                <td className="whitespace-nowrap px-4 py-3 font-mono text-xs text-muted-foreground">{p.reference}</td>
                                <td className="whitespace-nowrap px-4 py-3">
                                    <div className="flex items-center gap-2">
                                        <img src={p.image_url} alt="" className="w-8 h-8 rounded object-cover" />
                                        <span className="font-medium max-w-[180px] truncate">{p.name}</span>
                                    </div>
                                </td>
                                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">{p.categories?.name || ''}</td>
                                <td className="whitespace-nowrap px-4 py-3 font-bold text-primary">{formatPrice(p.price)}</td>
                                <td className="whitespace-nowrap px-4 py-3 text-center">{p.stock_quantity}</td>
                                <td className="whitespace-nowrap px-4 py-3">
                                    <Badge variant="outline" className={`text-xs border ${stockBadgeClass(status)} `>{stockLabel(status)}</td>
                                <td className="whitespace-nowrap px-4 py-3">
                                    <div className="flex items-center gap-1">
                                        <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => openEdit(p)}><PencilIcon />
                                        <Button variant="ghost" size="icon" className="h-8 w-8 text-destructive hover:text-destructive/80" onClick={() => openDelete(p)}><TrashIcon />
                                    </div>
                                </td>
                            </tr>
                        );
                    })}
                </tbody>
            </table>
        </div>
    </AdminLayout>
);

```

```

        </table>
    </div>
</div>

{/* Create/Edit Dialog */}
<Dialog open={dialogOpen} onOpenChange={setDialogOpen}>
    <DialogContent className="max-w-[calc(100%-2rem)] md:max-w-xl max-h-[90dvh] overflow-y-auto">
        <DialogHeader>
            <DialogTitle>{editingId ? 'Modifier le produit' : 'Nouveau produit'}</DialogTitle>
        </DialogHeader>
        <div className="space-y-4">
            <div className="grid sm:grid-cols-2 gap-4">
                <div className="space-y-2">
                    <Label>Nom </Label>
                    <Input value={form.name} onChange={(e) => setForm({ ...form, name: e.target.value })} placeholder="Nom du produit" />
                </div>
                <div className="space-y-2">
                    <Label>Référence </Label>
                    <Input value={form.reference} onChange={(e) => setForm({ ...form, reference: e.target.value })} placeholder="Référence du produit" />
                </div>
            </div>
            <div className="space-y-2">
                <Label>Catégorie </Label>
                <Select value={form.category_id} onChange={(v) => setForm({ ...form, category_id: v })}>
                    <SelectTrigger><SelectValue placeholder="Choisir une catégorie" /></SelectTrigger>
                    <SelectContent>
                        {categories.map((c) => (
                            <SelectItem key={c.id} value={c.id}>{c.type === 'electricite' ? '■' : '■■'} {c.name}</SelectItem>
                        ))}
                    </SelectContent>
                </Select>
            </div>
            <div className="space-y-2">
                <Label>Description</Label>
                <Textarea value={form.description} onChange={(e) => setForm({ ...form, description: e.target.value })} />
            </div>
            <div className="grid grid-cols-3 gap-4">
                <div className="space-y-2">
                    <Label>Prix ($) </Label>
                    <Input type="number" min="0" step="0.01" value={form.price} onChange={(e) => setForm({ ...form, price: e.target.value })} />
                </div>
                <div className="space-y-2">
                    <Label>Stock</Label>
                    <Input type="number" min="0" value={form.stock_quantity} onChange={(e) => setForm({ ...form, stock_quantity: e.target.value })} />
                </div>
                <div className="space-y-2">
                    <Label>Seuil alerte</Label>
                    <Input type="number" min="0" value={form.stock_threshold} onChange={(e) => setForm({ ...form, stock_threshold: e.target.value })} />
                </div>
            </div>
            <div className="space-y-2">
                <Label>URL image</Label>
                <Input value={form.image_url} onChange={(e) => setForm({ ...form, image_url: e.target.value })} placeholder="URL de l'image du produit" />
            </div>
        </div>
        <DialogFooter className="gap-2">
            <Button variant="outline" onClick={() => setDialogOpen(false)}>Annuler</Button>
            <Button onClick={handleSave} disabled={saving} className="bg-primary hover:bg-primary/90 text-primary-foreground">
                {saving ? <Loader2 className="w-4 h-4 animate-spin mr-2" /> : 'Enregistrer'}
            </Button>
        </DialogFooter>
    </DialogContent>
</Dialog>

{/* Delete confirmation */}
<AlertDialog open={!deleteId} onOpenChange={(o) => { if (!o) setDeleteId(null); }}>
    <AlertDialogContent className="max-w-[calc(100%-2rem)] md:max-w-lg">
        <AlertDialogHeader>
            <AlertDialogTitle>Confirmer la suppression</AlertDialogTitle>
            <AlertDialogDescription>Êtes-vous sûr de vouloir supprimer ce produit ? Cette action est irréversible.</AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
            <Button variant="outline" onClick={() => setDeleteId(null)}>Annuler</Button>
            <Button variant="destructive" onClick={handleDelete}>Supprimer</Button>
        </AlertDialogFooter>
    </AlertDialogContent>
</AlertDialog>

```

```

        <AlertDialogTitle>Supprimer ce produit ?</AlertDialogTitle>;
        <AlertDialogDescription>Cette action est irréversible.</AlertDialogDescription>;
    </AlertDialogHeader>;
    <AlertDialogFooter>
        <AlertDialogCancel>Annuler</AlertDialogCancel>;
        <AlertDialogAction onClick={handleDelete} className="bg-destructive text-destructive-foreground hover:bg-des
        </AlertDialogFooter>;
    </AlertDialogContent>;
    </AlertDialog>;
    </AdminLayout>;
  );
};

export default AdminProducts;

```

src/pages/admin/AdminStock.tsx

162 lignes | 7,774 octets

```

import React, { useEffect, useState, useCallback } from 'react';
import { Package, AlertTriangle, XCircle, CheckCircle2, Search, X } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Badge } from '@components/ui/badge';
import { Card, CardContent } from '@components/ui/card';
import AdminLayout from '../AdminLayout';
import { fetchAllProducts } from '@services/api';
import { supabase } from '@db/supabase';
import { getStockStatus, stockLabel, stockBadgeClass, formatPrice } from '@lib/helpers';
import type { Product } from '@types/index';
import { toast } from 'sonner';

const AdminStock: React.FC = () => {
  const [products, setProducts] = useState<Product[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [filter, setFilter] = useState<'all' | 'en_stock' | 'stock_faible' | 'rupture'>('all');
  const [editing, setEditing] = useState<Record<string, string>>({});
  const [saving, setSaving] = useState<string | null>(null);

  const load = useCallback(() => {
    setLoading(true);
    fetchAllProducts().then(setProducts).catch(console.error).finally(() => setLoading(false));
  }, []);

  useEffect(() => { load(); }, [load]);

  const filtered = products.filter((p) => {
    const status = getStockStatus(p);
    const matchFilter = filter === 'all' || status === filter;
    const matchSearch = !search.trim() || p.name.toLowerCase().includes(search.toLowerCase()) || p.reference.toLowerCase().includes(search.toLowerCase());
    return matchFilter & matchSearch;
  });

  const counts = {
    all: products.length,
    en_stock: products.filter((p) => getStockStatus(p) === 'en_stock').length,
    stock_faible: products.filter((p) => getStockStatus(p) === 'stock_faible').length,
    rupture: products.filter((p) => getStockStatus(p) === 'rupture').length,
  };

  const handleStockChange = (id: string, val: string) => {
    setEditing({ ...editing, [id]: val });
  };

  const handleStockSave = async (product: Product) => {

```

```

const newQty = parseInt(editing[product.id] ?? String(product.stock_quantity));
if (isNaN(newQty) || newQty < 0) { toast.error('Quantité invalide'); return; }
setSaving(product.id);
try {
  const { error } = await supabase.from('products').update({ stock_quantity: newQty, updated_at: new Date().toISOString() });
  if (error) throw error;
  toast.success(`Stock mis à jour : ${product.name}`);
  load();
  setEditing((prev) => { const n = { ...prev }; delete n[product.id]; return n; });
} catch (err) {
  console.error(err);
  toast.error('Erreur lors de la mise à jour');
} finally {
  setSaving(null);
}
};

const filterButtons: Array<{ key: keyof filter; label: string; icon: React.ElementType; color: string }> = [
  { key: 'all', label: 'Tous', icon: Package, color: 'text-foreground' },
  { key: 'en_stock', label: 'En stock', icon: CheckCircle2, color: 'text-green-500' },
  { key: 'stock_faible', label: 'Stock faible', icon: AlertTriangle, color: 'text-yellow-500' },
  { key: 'rupture', label: 'Rupture', icon: XCircle, color: 'text-destructive' },
];

return (
  <AdminLayout>
    <div className="space-y-5">
      <div>
        <h1 className="text-2xl font-bold">Gestion du stock</h1>
        <p className="text-muted-foreground text-sm">Suivez et mettez à jour les niveaux de stock</p>
      </div>

      { /* Summary cards */ }
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        {filterButtons.map(({ key, label, icon: Icon, color }) => (
          <Card
            key={key}
            className={`border cursor-pointer transition-colors ${filter === key ? 'border-primary bg-primary/5' : 'border-transparent'}`}
            onClick={() => setFilter(key)}
          >
            <CardContent className="p-4 flex items-center gap-3">
              <Icon className={`w-5 h-5 ${color}`} />
              <div>
                <p className="text-xs text-muted-foreground">{label}</p>
                <p className={`text-xl font-bold ${color}`}>{counts[key]}</p>
              </div>
            </CardContent>
          </Card>
        ))}
      </div>

      { /* Search */ }
      <div className="relative max-w-sm">
        <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
        <Input placeholder="Rechercher..." value={search} onChange={(e) => setSearch(e.target.value)} className="pl-10" />
        <button className="absolute right-3 top-1/2 -translate-y-1/2" onClick={() => setSearch('')} />
      </div>

      { /* Table */ }
      <div className="w-full max-w-full overflow-x-auto bg-card rounded-lg border border-border">
        <table className="w-full min-w-max text-sm">
          <thead>
            <tr className="border-b border-border">
              {['Réf', 'Produit', 'Prix', 'Stock actuel', 'Seuil', 'État', 'Modifier'].map((h) => (
                <th key={h} className="whitespace-nowrap text-left px-4 py-3 text-xs font-semibold text-muted-foreground">{h}</th>
              ))}
            </tr>
          </thead>
        </table>
      </div>
    </div>
  </AdminLayout>
);

```

```

    <tbody>
      {loading ? (
        <tr>
          <td colspan={7} className="text-center py-10 text-muted-foreground">Chargement...</td>
        ) : filtered.map((p) => {
          const status = getStockStatus(p);
          const editVal = editing[p.id] ?? String(p.stock_quantity);
          return (
            <tr key={p.id} className="border-b border-border/50 hover:bg-muted/30 transition-colors">
              <td className="whitespace-nowrap px-4 py-3 font-mono text-xs text-muted-foreground">{p.reference}</td>
              <td className="whitespace-nowrap px-4 py-3 font-medium max-w-[200px] truncate">{p.name}</td>
              <td className="whitespace-nowrap px-4 py-3 text-primary font-bold">{formatPrice(p.price)}</td>
              <td className="whitespace-nowrap px-4 py-3 text-center font-semibold">{p.stock_quantity}</td>
              <td className="whitespace-nowrap px-4 py-3 text-center text-muted-foreground">{p.stock_threshold}</td>
              <td className="whitespace-nowrap px-4 py-3">
                <Badge variant="outline" className={`text-xs border ${stockBadgeClass(status)}`}>{stockLabel(status)}</td>
              <td className="whitespace-nowrap px-4 py-3">
                <div className="flex items-center gap-2">
                  <Input
                    type="number"
                    min="0"
                    value={editVal}
                    onChange={(e) => handleStockChange(p.id, e.target.value)}
                    className="w-20 h-8 text-center px-2"
                  />
                  <Button
                    size="sm"
                    className="h-8 bg-primary hover:bg-primary/90 text-primary-foreground"
                    onClick={() => handleStockSave(p)}
                    disabled={saving === p.id}
                  />
                </div>
                {saving === p.id ? '...' : 'OK'}
              </td>
            </tr>
          );
        })
      </tbody>
    </table>
  </div>
</div>
</AdminLayout>
);
};
export default AdminStock;

```

[src/pages/admin/AdminOrders.tsx](#)

189 lignes | 9,612 octets

```

import React, { useEffect, useState, useCallback } from 'react';
import { Search, X, FileDown, Eye } from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Badge } from '@/components/ui/badge';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@/components/ui/select';
import {
  Dialog, DialogContent, DialogHeader, DialogTitle
} from '@/components/ui/dialog';
import AdminLayout from '../AdminLayout';
import { fetchOrders, updateOrderStatus, fetchInvoiceByOrderId } from '@/services/api';
import { formatDate, formatPrice, orderStatusLabel } from '@/lib/helpers';
import { generateInvoicePDF } from '@/lib/pdfGenerator';
import type { Order } from '@/types/index';
import { toast } from 'sonner';

```

```

const statusColors: Record<string, string> = {
  en_attente: 'text-yellow-500 border-yellow-500/30 bg-yellow-500/5',
  confirme: 'text-blue-500 border-blue-500/30 bg-blue-500/5',
  en_preparation: 'text-accent border-accent/30 bg-accent/5',
  livre: 'text-success border-success/30 bg-success/5',
  annule: 'text-destructive border-destructive/30 bg-destructive/5',
};

const AdminOrders: React.FC = () => {
  const [orders, setOrders] = useState<Order[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [selected, setSelected] = useState<Order | null>(null);
  const [pdfLoading, setPdfLoading] = useState<string | null>(null);

  const load = useCallback(() => {
    setLoading(true);
    fetchOrders(100).then(setOrders).catch(console.error).finally(() => setLoading(false));
  }, []);

  useEffect(() => { load(); }, [load]);

  const filtered = orders.filter((o) =>
    !search.trim() ||
    o.order_number.toLowerCase().includes(search.toLowerCase()) ||
    (o.clients?.full_name || '').toLowerCase().includes(search.toLowerCase())
  );

  const handleStatusChange = async (order: Order, status: Order['status']) => {
    try {
      await updateOrderStatus(order.id, status);
      toast.success('Statut mis à jour');
      load();
      if (selected?.id === order.id) setSelected({ ...selected, status });
    } catch (err) {
      console.error(err);
      toast.error('Erreur');
    }
  };

  const handleDownloadInvoice = async (orderId: string) => {
    setPdfLoading(orderId);
    try {
      const invoice = await fetchInvoiceByOrderId(orderId);
      if (!invoice) { toast.error('Facture introuvable'); return; }
      generateInvoicePDF(invoice);
    } catch (err) {
      console.error(err);
      toast.error('Erreur lors de la génération PDF');
    } finally {
      setPdfLoading(null);
    }
  };

  return (
    <AdminLayout>
      <div className="space-y-5">
        <div>
          <h1 className="text-2xl font-bold">Commandes</h1>
          <p className="text-muted-foreground text-sm">{orders.length} commandes</p>
        </div>

        <div className="relative max-w-sm">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
          <Input placeholder="Rechercher par n° ou client..." value={search} onChange={(e) => setSearch(e.target.value)}
            {search & & <button onClick={() => setSearch('')} className="absolute right-3 top-1/2 -translate-y-1/2" />
          </div>
          <div className="w-full max-w-full overflow-x-auto bg-card rounded-lg border border-border">

```

```

<table className="w-full min-w-max text-sm">
  <thead>
    <tr className="border-b border-border">
      {['N° Commande', 'Client', 'Date', 'Mode', 'Total', 'Statut', 'Actions'].map((h) => (
        <th key={h} className="whitespace-nowrap text-left px-4 py-3 text-xs font-semibold text-muted-foreground">
          {h}
        </th>
      ))}
    </tr>
  </thead>
  <tbody>
    {loading ? (
      <tr>
        <td colspan={7} className="text-center py-10 text-muted-foreground">Chargement...</td>
      </tr>
    ) : filtered.length === 0 ? (
      <tr>
        <td colspan={7} className="text-center py-10 text-muted-foreground">Aucune commande</td>
      </tr>
    ) : filtered.map((o) => (
      <tr key={o.id} className="border-b border-border/50 hover:bg-muted/30 transition-colors">
        <td className="whitespace-nowrap px-4 py-3 font-mono text-xs">{o.order_number}</td>
        <td className="whitespace-nowrap px-4 py-3 font-medium">{o.clients?.full_name || '-'}</td>
        <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">{formatDate(o.created_at)}</td>
        <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">
          {o.delivery_mode === 'livraison' ? '■ Livraison' : '■ Retrait'}
        </td>
        <td className="whitespace-nowrap px-4 py-3 font-bold text-primary">{formatPrice(o.total)}</td>
        <td className="whitespace-nowrap px-4 py-3">
          <Select value={o.status} onChange={(v) => {handleStatusChange(o, v as Order['status'])}}>
            <SelectTrigger className={`h-7 text-xs w-36 border ${statusColors[o.status] || ''}`>
              <SelectValue />
            </SelectTrigger>
            <SelectContent>
              {['en_attente', 'confirme', 'en_preparation', 'livre', 'annule'].map((s) => (
                <SelectItem key={s} value={s}>{orderStatusLabel(s)}</SelectItem>
              ))}
            </SelectContent>
          </Select>
        </td>
        <td className="whitespace-nowrap px-4 py-3">
          <div className="flex gap-1">
            <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => setSelected(o)}><Icon />
            <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => handleDownloadInvoice(o)}><Icon />
            <FileDown className={`w-3.5 h-3.5 ${pdfLoading === o.id ? 'animate-bounce' : ''}`} />
          </div>
        </td>
      </tr>
    ))}
  </tbody>
</table>
</div>

/* Order detail dialog */
<Dialog open={!selected} onOpenChange={(o) => {if (!o) setSelected(null);}}>
  <DialogContent className="max-w-[calc(100%-2rem)] md:max-w-xl max-h-[90dvh] overflow-y-auto">
    <DialogHeader>
      <DialogTitle>Commande {selected?.order_number}</DialogTitle>
    </DialogHeader>
    {selected && (
      <div className="space-y-4 text-sm">
        <div className="grid grid-cols-2 gap-3">
          <div>
            <p>Client</p>
            <p>Téléphone</p>
          </div>
          <div>
            <p>Date</p>
            <p>Mode</p>
          </div>
          <div>
            <p>Statut</p>
            <p>Adresse de livraison</p>
          </div>
        </div>
        <div>
          <p>Total</p>
          <p>Statut</p>
          <p>Actions</p>
        </div>
      </div>
    )}
  </DialogContent>
</Dialog>

```

```

        <tr>
          <th className="text-left px-3 py-2">Produit</th>
          <th className="text-center px-3 py-2">Qté</th>
          <th className="text-right px-3 py-2">Total</th>
        </tr>
      </thead>
      <tbody>
        {(selected.order_items || []).map((item) => (
          <tr key={item.id} className="border-t border-border/50">
            <td className="px-3 py-2">{item.product_name}</td>
            <td className="text-center px-3 py-2">{item.quantity}</td>
            <td className="text-right px-3 py-2 font-medium">{formatPrice(item.total)}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>

  <div className="space-y-1 text-right">
    <p className="text-muted-foreground">Sous-total : {formatPrice(selected.subtotal)}</p>
    <p className="text-muted-foreground">Livraison : {formatPrice(selected.delivery_fee)}</p>
    <p className="font-bold text-lg text-primary">Total : {formatPrice(selected.total)}</p>
  </div>

  <Button className="w-full gap-2 bg-primary hover:bg-primary/90 text-primary-foreground" onClick={() => {
    <FileDown className="w-4 h-4" />Télécharger la facture PDF
  }}>
  </div>
)}
</DialogContent>
</Dialog>
</AdminLayout>
);
};

export default AdminOrders;

```

src/pages/admin/AdminInvoices.tsx

119 lignes | 6,317 octets

```

import React, { useEffect, useState, useCallback } from 'react';
import { Search, X, Eye, FileDown, Loader2 } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@components/ui/dialog';
import AdminLayout from '../AdminLayout';
import { fetchInvoices, fetchInvoiceById } from '@services/api';
import { formatDate, formatPrice } from '@lib/helpers';
import { generateInvoicePDF } from '@lib/pdfGenerator';
import type { Invoice } from '@types/index';
import { toast } from 'sonner';

const AdminInvoices: React.FC = () => {
  const [invoices, setInvoices] = useState<Invoice[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [selected, setSelected] = useState<Invoice | null>(null);
  const [pdfLoading, setPdfLoading] = useState<string | null>(null);

  const load = useCallback(() => {
    setLoading(true);
    fetchInvoices(100).then(setInvoices).catch(console.error).finally(() => setLoading(false));
  }, []);

  useEffect(() => { load(); }, [load]);

```

```

const filtered = invoices.filter((inv) => {
  !search.trim() ||
  inv.invoice_number.toLowerCase().includes(search.toLowerCase()) ||
  (inv.clients?.full_name || '').toLowerCase().includes(search.toLowerCase())
});

const handleDownload = async (invoice: Invoice) => {
  setPdfLoading(invoice.id);
  try {
    const full = await fetchInvoiceById(invoice.order_id);
    if (!full) { toast.error('Facture introuvable'); return; }
    generateInvoicePDF(full);
  } catch (err) {
    console.error(err);
    toast.error('Erreur PDF');
  } finally {
    setPdfLoading(null);
  }
};

return (
  <AdminLayout>
    <div className="space-y-5">
      <div>
        <h1 className="text-2xl font-bold">Factures</h1>
        <p className="text-muted-foreground text-sm">{invoices.length} factures générées</p>
      </div>

      <div className="relative max-w-sm">
        <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
        <input placeholder="Rechercher..." value={search} onChange={(e) => setSearch(e.target.value)} className="pl-10" />
        <button onClick={() => setSearch('')} className="absolute right-3 top-1/2 -translate-y-1/2" />
      </div>

      <div className="w-full max-w-full overflow-x-auto bg-card rounded-lg border border-border">
        <table className="w-full min-w-max text-sm">
          <thead>
            <tr className="border-b border-border">
              {[ 'N° Facture', 'Client', 'Commande', 'Date', 'Total', 'Actions' ].map((h) => (
                <th key={h} className="whitespace-nowrap text-left px-4 py-3 text-xs font-semibold text-muted-foreground">{h}</th>
              ))}
            </tr>
          </thead>
          <tbody>
            {loading ? (
              <tr>
                <td colspan={6} className="text-center py-10"><Loader2 className="w-5 h-5 animate-spin m-auto" /></td>
            ) : filtered.length === 0 ? (
              <tr>
                <td colspan={6} className="text-center py-10 text-muted-foreground">Aucune facture</td>
            ) : filtered.map((inv) => (
              <tr key={inv.id} className="border-b border-border/50 hover:bg-muted/30 transition-colors">
                <td className="whitespace-nowrap px-4 py-3 font-mono text-xs font-semibold text-primary">{inv.invoice_number}</td>
                <td className="whitespace-nowrap px-4 py-3 font-medium">{inv.clients?.full_name || '-'}</td>
                <td className="whitespace-nowrap px-4 py-3 font-mono text-xs text-muted-foreground">{inv.orders?.order_number}</td>
                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">{formatDate(inv.issued_at)}</td>
                <td className="whitespace-nowrap px-4 py-3 font-bold text-primary">{formatPrice(inv.total)}</td>
                <div className="flex gap-1">
                  <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => setSelected(inv)}></Button>
                  <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => handleDownload(inv)}></Button>
                  {pdfLoading === inv.id ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <FileDownload className="h-8 w-8" />}
                </div>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  </AdminLayout>
);

```

```

    </div>;

    <Dialog open={!selected} onOpenChange={{(o) => { if (!o) setSelected(null); }}>
      <DialogContent className="max-w-[calc(100%-2rem)] md:max-w-md max-h-[90dvh] overflow-y-auto">
        <DialogHeader><DialogTitle>Facture {selected?.invoice_number}</DialogTitle></DialogHeader>
        {selected &amp; &amp; (
          <div className="space-y-3 text-sm">
            <div className="grid grid-cols-2 gap-2">
              <div><p className="text-xs text-muted-foreground">Client</p><p className="font-medium">
                <div><p className="text-xs text-muted-foreground">Date</p><p className="font-medium">
                  <div><p className="text-xs text-muted-foreground">N° Commande</p><p className="font-medium">
                    <div><p className="text-xs text-muted-foreground">Total</p><p className="font-bold text">
                      <Button className="w-full gap-2 bg-primary hover:bg-primary/90 text-primary-foreground" onClick={() => {
                        <FileDown className="w-4 h-4" />Télécharger PDF
                      </Button>
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </DialogContent>
        </Dialog>
      </AdminLayout>
    );
  };

  export default AdminInvoices;

```

src/pages/admin/AdminQuotes.tsx

144 lignes | 7,540 octets

```

import React, { useEffect, useState, useCallback } from 'react';
import { Search, X, Eye, Loader2 } from 'lucide-react';
import { Button } from '@components/ui/button';
import { Input } from '@components/ui/input';
import { Badge } from '@components/ui/badge';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@components/ui/select';
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@components/ui/dialog';
import AdminLayout from '../AdminLayout';
import { fetchQuotes, updateQuoteStatus } from '@services/api';
import { formatDate, quoteStatusLabel } from '@lib/helpers';
import type { Quote } from '@types/index';
import { toast } from 'sonner';

const statusColors: Record<string, string> = {
  nouveau: 'text-blue-500 border-blue-500/30 bg-blue-500/5',
  en_cours: 'text-yellow-500 border-yellow-500/30 bg-yellow-500/5',
  traite: 'text-success border-success/30 bg-success/5',
  rejete: 'text-destructive border-destructive/30 bg-destructive/5',
};

const AdminQuotes: React.FC = () => {
  const [quotes, setQuotes] = useState<Quote[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [selected, setSelected] = useState<Quote | null>(null);

  const load = useCallback(() => {
    setLoading(true);
    fetchQuotes(100).then(setQuotes).catch(console.error).finally(() => setLoading(false));
  }, []);

  useEffect(() => { load(); }, [load]);

  const filtered = quotes.filter((q) =>
    !search.trim() ||
    q.full_name.toLowerCase().includes(search.toLowerCase()) ||

```

```

    q.project_type.toLowerCase().includes(search.toLowerCase())
  );

const handleStatus = async (id: string, status: Quote['status']) => {
  try {
    await updateQuoteStatus(id, status);
    toast.success('Statut mis à jour');
    load();
  } catch (err) { toast.error('Erreur'); }
};

return (
  <AdminLayout>
    <div className="space-y-5">
      <div>
        <h1 className="text-2xl font-bold">Demandes de devis</h1>
        <p className="text-muted-foreground text-sm">{quotes.length} demandes</p>
      </div>

      <div className="relative max-w-sm">
        <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
        <Input placeholder="Rechercher..." value={search} onChange={(e) => setSearch(e.target.value)} className="pl-10" />
        <button onClick={() => setSearch('')} className="absolute right-3 top-1/2 -translate-y-1/2" />
      </div>

      <div className="w-full max-w-full overflow-x-auto bg-card rounded-lg border border-border">
        <table className="w-full min-w-max text-sm">
          <thead>
            <tr className="border-b border-border">
              {[ 'N° Devis', 'Client', 'Téléphone', 'Type de projet', 'Date', 'Statut', 'Actions' ].map((h) => (
                <th key={h} className="whitespace-nowrap text-left px-4 py-3 text-xs font-semibold text-muted-foreground">{h}</th>
              ))}
            </tr>
          </thead>
          <tbody>
            {loading ? (
              <tr>
                <td colspan={7} className="text-center py-10"><Loader2 className="w-5 h-5 animate-spin" />
              </td>
            ) : filtered.map((q) => (
              <tr key={q.id} className="border-b border-border/50 hover:bg-muted/30 transition-colors">
                <td className="whitespace-nowrap px-4 py-3 font-mono text-xs">{q.quote_number || '-'}</td>
                <td className="whitespace-nowrap px-4 py-3 font-medium">{q.full_name}</td>
                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">{q.phone}</td>
                <td className="whitespace-nowrap px-4 py-3 text-xs max-w-[160px] truncate">{q.project_type}</td>
                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">{formatDate(q.created_at)}</td>
                <td className="whitespace-nowrap px-4 py-3">
                  <Select value={q.status} onChange={(v) => handleStatus(q.id, v as Quote['status'])}>
                    <SelectTrigger className="h-7 text-xs w-32 border ${statusColors[q.status] || ''}">
                      <SelectValue />
                    </SelectTrigger>
                    <SelectContent>
                      {[ 'nouveau', 'en_cours', 'traite', 'rejete' ].map((s) => (
                        <SelectItem key={s} value={s}>{quoteStatusLabel(s)}</SelectItem>
                      ))}
                    </SelectContent>
                  </td>
                <td className="whitespace-nowrap px-4 py-3">
                  <Button variant="ghost" size="icon" className="h-8 w-8" onClick={() => setSelected(q)}><Eye />
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>

      <Dialog open={!selected} onOpenChange={(o) => { if (!o) setSelected(null); }}>
        <DialogContent className="max-w-[calc(100%-2rem)] md:max-w-lg max-h-[90dvh] overflow-y-auto">

```

```

</DialogHeader><DialogTitle>Devis - {selected?.full_name}</DialogTitle></DialogHeader>
{selected && (
  <div className="space-y-4 text-sm">
    <div className="grid grid-cols-2 gap-3">
      <div><p className="text-xs text-muted-foreground">Nom</p><p className="font-medium">
        <div><p className="text-xs text-muted-foreground">Téléphone</p><p className="font-medium">
          {selected.email && <div><p className="text-xs text-muted-foreground">Email</p><p>
            <div><p className="text-xs text-muted-foreground">Type</p><p className="font-medium">
              {selected.address && <div className="col-span-2"><p className="text-xs text-muted-foreground">
                </div>
            </div><p className="text-xs text-muted-foreground mb-1">Description</p>
            <p className="text-sm bg-muted/40 rounded-lg p-3 leading-relaxed">{selected.description}</p>
          </div>
          {selected.photo_urls && selected.photo_urls.length > 0 && (
            <div>
              <p className="text-xs text-muted-foreground mb-2">Photos ({selected.photo_urls.length})</p>
              <div className="grid grid-cols-3 gap-2">
                {selected.photo_urls.map((url, i) => (
                  <a key={i} href={url} target="_blank" rel="noreferrer">
                    <img src={url} alt={`Photo ${i + 1}`} className="w-full aspect-square object-cover rounded-md" />
                  </a>
                ))}
              </div>
            </div>
          </div>
        )}
      </div>
      <div>
        <p className="text-xs text-muted-foreground mb-1">Statut</p>
        <Badge variant="outline" className={`text-xs border ${statusColors[selected.status] || ''}`>
          {quoteStatusLabel(selected.status)}
        </Badge>
      </div>
    </div>
  )}
</div>
</DialogContent>
</Dialog>
</AdminLayout>
);
};

export default AdminQuotes;

```

src/pages/admin/AdminClients.tsx

73 lignes | 3,488 octets

```

import React, { useEffect, useState, useCallback } from 'react';
import { Search, X, Loader2 } from 'lucide-react';
import { Input } from '@components/ui/input';
import AdminLayout from '../AdminLayout';
import { fetchClients } from '@services/api';
import { formatDate } from '@lib/helpers';
import type { Client } from '@types/index';

const AdminClients: React.FC = () => {
  const [clients, setClients] = useState<Client[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');

  const load = useCallback(() => {
    setLoading(true);
    fetchClients(200).then(setClients).catch(console.error).finally(() => setLoading(false));
  }, []);

  useEffect(() => { load(); }, [load]);

  const filtered = clients.filter((c) =>

```

```

!search.trim() ||
c.full_name.toLowerCase().includes(search.toLowerCase()) ||
c.phone.includes(search) ||
(c.email || '').toLowerCase().includes(search.toLowerCase())
);

return (
  <AdminLayout>
    <div className="space-y-5">
      <div>
        <h1 className="text-2xl font-bold">Clients</h1>
        <p className="text-muted-foreground text-sm">{clients.length} clients enregistrés</p>
      </div>

      <div className="relative max-w-sm">
        <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
        <Input placeholder="Rechercher par nom, téléphone ou email..." value={search} onChange={(e) => setSearch(e.target.value)} />
        <button onClick={() => setSearch('')} className="absolute right-3 top-1/2 -translate-y-1/2" />
      </div>

      <div className="w-full max-w-full overflow-x-auto bg-card rounded-lg border border-border">
        <table className="w-full min-w-max text-sm">
          <thead>
            <tr className="border-b border-border">
              <th key={h} className="whitespace-nowrap text-left px-4 py-3 text-xs font-semibold text-muted-foreground">{h}</th>
            </tr>
          </thead>
          <tbody>
            {loading ? (
              <tr>
                <td colspan={5} className="text-center py-10"><Loader2 className="w-5 h-5 animate-spin" /></td>
              </tr>
            ) : filtered.length === 0 ? (
              <tr>
                <td colspan={5} className="text-center py-10 text-muted-foreground">Aucun client trouvé</td>
              </tr>
            ) : filtered.map((c) => (
              <tr key={c.id} className="border-b border-border/50 hover:bg-muted/30 transition-colors">
                <td className="whitespace-nowrap px-4 py-3 font-medium">{c.full_name}</td>
                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground">{c.phone}</td>
                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">{c.email || '-'}</td>
                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs max-w-[180px] truncate">{c.address}</td>
                <td className="whitespace-nowrap px-4 py-3 text-muted-foreground text-xs">{formatDate(c.created_at)}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </AdminLayout>
  );
};

export default AdminClients;

```